PREMIUM HANDWRITTEN INTERVIEW NOTEBOOK

AWS

Zero to Hero

Cloud architecture, explained with sketches, Java examples and safe practice labs.

100 ORIGINAL QUESTIONS | 100 ANSWERED FOLLOW-UPS | 20 CHAPTERS | 8 PRACTICE LABS

```
client → ALB → Spring tasks
order DB + outbox → SQS
Know the contract. Defend the failure.
```

Java & Spring Boot focus | October 2026

START HERE

Learn it. Draw it. Explain it.

A beginner-friendly AWS interview notebook that keeps the original questions but gives you a shorter spoken answer, a diagram and an actual follow-up answer.

100	100	08
Original questions, unchanged	Distinct follow-up answers	Cloud-read-only and offline exercises

Study routine: Give the short answer in 20–30 seconds. Trace the three-box diagram. Explain the provided AWS example. Name a failure boundary. Then answer the follow-up without looking.

Read carefully: Source examples use placeholders and are not universal deployment instructions. Service features and AWS prices change. Check official docs for your selected Region and service mode.

What was tested: Four Python-based offline labs were executed successfully, and all eight scripts passed Bash syntax validation. No AWS API was contacted. The Java AWS SDK example is a source template, not compiled here.

The cloud exercises L01–L04 only inspect authorized existing resources. Never paste real access keys into the examples. Keep IAM policies least-privileged.

COMPLETE ROADMAP

20 chapters from beginner to architect

01 Cloud foundations and interview thinking

02 IAM, STS and authorization

03 VPC and network controls

04 EC2, Auto Scaling and instance operations

05 S3 and storage choices

06 Load balancing, DNS and edge delivery

07 ECS, EKS and Java containers

08 Lambda and API Gateway

09 RDS, Aurora and relational reliability

10 DynamoDB data modeling

11 Caching and ElastiCache

12 SQS, SNS and EventBridge

13 Streaming and workflow orchestration

14 Observability and incident diagnosis

15 KMS, secrets and application security

16 Infrastructure as code and CI/CD

17 Resilience, retries and disaster recovery

18 Cost, quotas and performance

19 Senior architecture scenarios

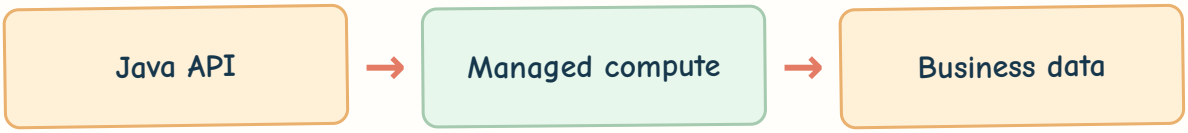
20 Troubleshooting and interview traps

Order: Chapters 01–06 build cloud fundamentals; 07–13 cover backend workload decisions; 14–20 cover reliability, delivery, cost and architect scenarios.

CHAPTER 01 / 20

Cloud foundations and interview thinking

Choose boundaries before technology names.



YOUR FIVE INTERVIEW QUESTIONS

001. What is AWS and what does cloud computing change?

002. What is the shared responsibility model?

003. What is the difference between a Region, Availability Zone and edge location?

004. Is high availability the same as disaster recovery?

005. How should a senior candidate approach an AWS design question?

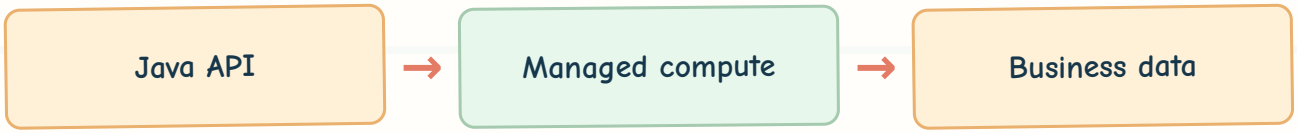
> Say the guarantee and its limit before describing any advanced service.

Q001. What is AWS and what does cloud computing change?

SAY THIS FIRST · SIMPLE ENGLISH

AWS lets teams rent computing, storage, and managed services when needed instead of buying all the hardware first.

Precise interview detail: AWS supplies on-demand infrastructure and managed services. You trade some infrastructure ownership for service contracts, usage-based costs and API-driven operations. Your application correctness, access controls and architecture still need deliberate engineering.



ORIGINAL AWS SCENARIO / COMMAND

```
Traditional: buy server -> install OS -> deploy app
AWS option: provision capacity -> deploy -> measure
Managed option: use service API -> manage data and access
```

WHAT HAPPENS · A Spring service can run on a VM, a managed container platform or a function. The programming model and operating responsibilities differ.

ORIGINAL INTERVIEWER FOLLOW-UP

Which responsibilities would you keep when moving a Java API from EC2 to ECS?

Answer: You still own Java code, application configuration, IAM access, data protection, monitoring, deployments, and recovery. ECS manages container scheduling, not business correctness.

Common mistake: Calling every managed service serverless hides meaningful capacity and billing differences.

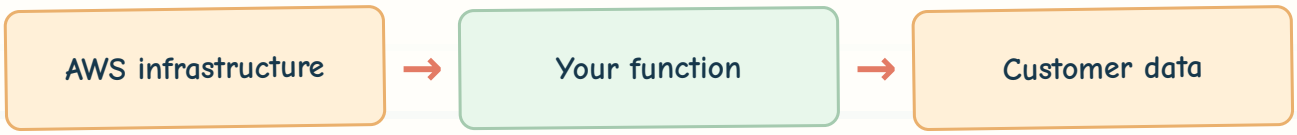
Source guide p. 6 · Further reading: A01, A02, A41, A42

Q002. What is the shared responsibility model?

SAY THIS FIRST · SIMPLE ENGLISH

AWS secures the cloud infrastructure. You secure the way your application uses cloud services and customer data.

Precise interview detail: AWS protects the underlying cloud infrastructure. Customers protect their use of it, including identities, data and configuration. The boundary moves by service: an EC2 guest OS needs your patching, while a managed database removes much of that host administration.



ORIGINAL AWS SCENARIO / COMMAND

```
EC2: customer patches guest OS and application
RDS: customer controls DB access, schema and queries
S3: customer controls object access and retention
```

WHAT HAPPENS · Using a managed service changes the boundary rather than eliminating your duties.

ORIGINAL INTERVIEWER FOLLOW-UP

How would you explain this boundary for Lambda?

Answer: For Lambda, AWS manages the hosts and runtime infrastructure; you still control function code, dependencies, IAM role, secrets, input checks, data, and observability.

Common mistake: An encrypted bucket can still expose data through an overly broad policy.

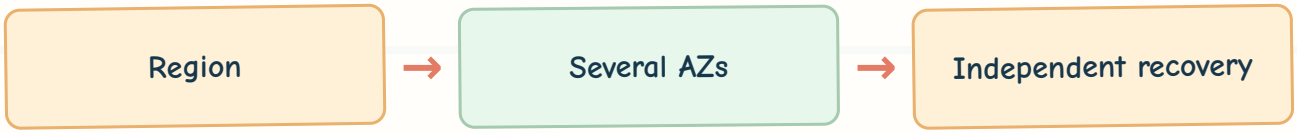
Source guide p. 7 · Further reading: A01, A02, A41, A42

Q003. What is the difference between a Region, Availability Zone and edge location?

SAY THIS FIRST · SIMPLE ENGLISH

A Region is a geographic area, an AZ is an isolated location inside a Region, and an edge location brings selected content closer to users.

Precise interview detail: A Region is a geographic AWS service area. An Availability Zone is an isolated infrastructure location within a Region, built from one or more data centers. Edge locations serve functions such as content delivery close to users; they are not interchangeable with application AZs.



ORIGINAL AWS SCENARIO / COMMAND

```
Region: application and primary data location
AZ A + AZ B: independent deployment failure domains
Edge: cache selected HTTP responses near users
```

WHAT HAPPENS · Choose Regions for latency, residency, service availability and disaster recovery. Choose AZs for in-Region resilience.

ORIGINAL INTERVIEWER FOLLOW-UP

When would two Regions be justified instead of two AZs?

Answer: Choose two Regions when a whole-Region outage, residency rule, or global latency target demands it. Multi-AZ is usually simpler for routine failure recovery within one Region.

Common mistake: Two instances in one AZ do not protect against that AZ failing.

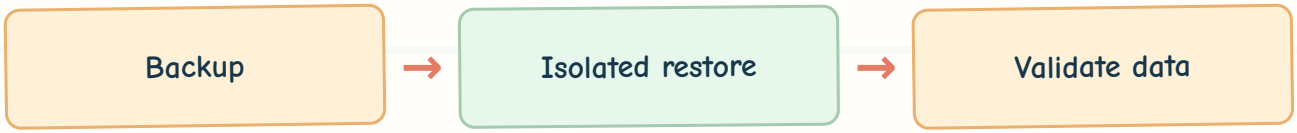
Source guide p. 7 · Further reading: A01, A02, A41, A42

Q004. Is high availability the same as disaster recovery?

SAY THIS FIRST · SIMPLE ENGLISH

High availability reduces ordinary outages. Disaster recovery gets the business running again after a major incident or data loss.

Precise interview detail: High availability limits interruption from expected component failures. Disaster recovery restores business capability after a larger disruption. Both require explicit failure assumptions; DR additionally depends on recovery procedures, independent copies and tested recovery objectives.



ORIGINAL AWS SCENARIO / COMMAND

```
HA scenario: one task or AZ fails
DR scenario: Region unavailable or data corrupted
RTO: time to recover service
RPO: acceptable amount of lost data, expressed as time
```

WHAT HAPPENS · A multi-AZ database can keep serving through a host failure while propagating accidental deletion to its replicas.

ORIGINAL INTERVIEWER FOLLOW-UP

How do you test recovery from a destructive schema migration?

Answer: Rehearse restoring a backup to a separate environment, apply or reverse migration steps, confirm key records and application compatibility, then measure RTO and RPO. Never test only backup creation.

Common mistake: Replica availability is not an independent recoverable backup.

Source guide p. 8 · Further reading: A01, A02, A41, A42

Q005. How should a senior candidate approach an AWS design question?

SAY THIS FIRST · SIMPLE ENGLISH

Start with requirements and risks, then pick AWS services. Tell the interviewer how you would test the design.

Precise interview detail: Clarify users, traffic, data, consistency, security and recovery objectives before choosing services. Explain an initial architecture, identify its bottleneck and failure boundary, and show how measurements would justify changes.



ORIGINAL AWS SCENARIO / COMMAND

```
Given: 500 requests/s, 200 ms target p95
Ask: peak factor, payload, transactions, residency
Design: ingress -> compute -> durable data
Validate: load, failure, restore, cost per request
```

WHAT HAPPENS · An interviewer learns more from your assumptions and tradeoffs than from a long service list.

ORIGINAL INTERVIEWER FOLLOW-UP

What new requirement would force you to change your first design?

Answer: A stricter recovery target, greater peak traffic, new data residency law, or critical cross-Region availability need can change the network, storage, or compute design. Say which assumption changed.

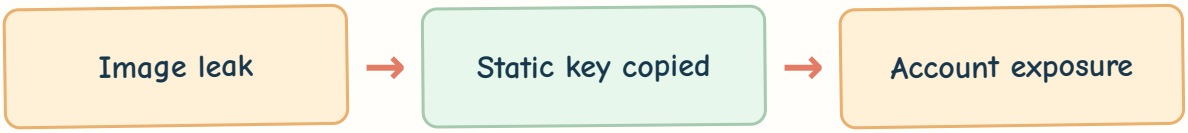
Common mistake: Treating illustrative traffic numbers as measured capacity leads to unsupported promises.

Source guide p. 8 · Further reading: A01, A02, A41, A42

CHAPTER 02 / 20

IAM, STS and authorization

Identify the real caller and effective authority.



YOUR FIVE INTERVIEW QUESTIONS

006. What are IAM users, roles and policies?

007. How does IAM evaluate access?

008. What is the difference between a trust policy and a permissions policy?

009. What do SCPs and permissions boundaries do?

010. What is least privilege for an application?

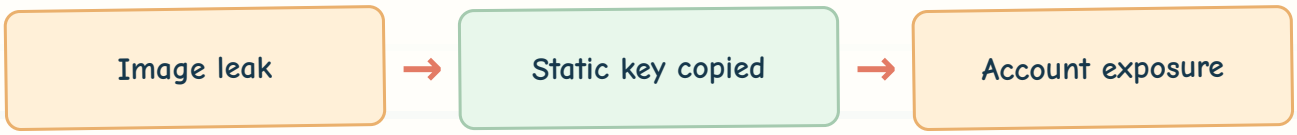
> Say the guarantee and its limit before describing any advanced service.

Q006. What are IAM users, roles and policies?

SAY THIS FIRST · SIMPLE ENGLISH

IAM users represent identities, roles offer assumable temporary identity, and policies say which API actions are allowed or denied.

Precise interview detail: A user is an identity that can have long-lived credentials. A role is an assumable identity normally used with temporary credentials. Policies describe permitted or denied actions. Prefer federation for people and workload roles for applications over distributing static access keys.



ORIGINAL AWS SCENARIO / COMMAND

```
Human -> identity provider -> temporary role session
ECS task -> task role -> S3 API
Policy -> Action + Resource + optional Condition
```

WHAT HAPPENS · The application uses a credential provider to obtain and refresh temporary credentials.

ORIGINAL INTERVIEWER FOLLOW-UP

Why are static keys especially risky inside a container image?

Answer: A leaked image or registry layer can expose a static key to anyone who can read it; rotation is hard once images are copied. Give containers short-lived workload-role credentials.

Common mistake: A role's existence does not grant every caller permission to assume it.

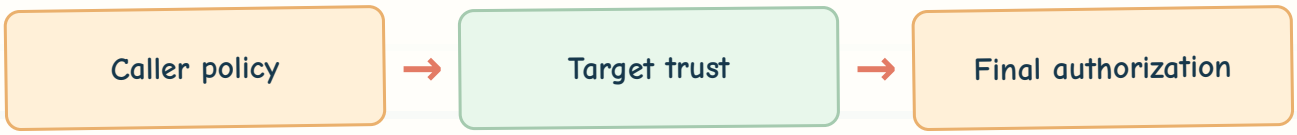
Source guide p. 9 · Further reading: A03-A06

Q007. How does IAM evaluate access?

SAY THIS FIRST · SIMPLE ENGLISH

IAM considers the caller, requested action, resource, and all applicable policies. An applicable explicit deny wins.

Precise interview detail: Authorization considers applicable identity and resource policies and relevant limiting controls. An applicable explicit deny takes precedence. Boundaries and organization policies limit authority rather than grant it. Evaluation details vary with the principal, account relationship and service.



ORIGINAL AWS SCENARIO / COMMAND

```
Identity Allow: s3:GetObject on reports/*
Boundary: permits the same action
Applicable explicit Deny: blocks the request
No relevant permission: implicit deny
```

WHAT HAPPENS · For diagnosis, identify the actual principal, API action, resource ARN and request context, then inspect every applicable control.

ORIGINAL INTERVIEWER FOLLOW-UP

What changes for a cross-account request?

Answer: Cross-account calls generally require permission on the caller side and trust or resource authorization on the target side, with SCPs and other controls also applied. Check both accounts and the actual role session.

Common mistake: Same-account resource grants to users, roles and sessions have different implicit-deny interactions; avoid a universal 'all policies intersect' rule.

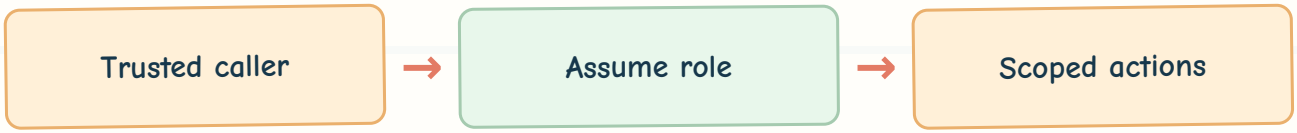
Source guide p. 10 · Further reading: A03-A06

Q008. What is the difference between a trust policy and a permissions policy?

SAY THIS FIRST · SIMPLE ENGLISH

A trust policy decides who can assume a role. A permissions policy decides what that role may do afterward.

Precise interview detail: A role trust policy controls who may assume the role and under what conditions. Its permissions policies control what the resulting role session may do. Cross-account access generally needs authorization in the caller account and trust or resource authorization in the target account.



ORIGINAL AWS SCENARIO / COMMAND

```
Trust: allow ecs-tasks.amazonaws.com to assume role
Permission: allow s3:GetObject on one object prefix
AssumeRole: returns temporary session credentials
```

WHAT HAPPENS · A trusted caller gets only the permissions of the session, subject to applicable restrictions.

ORIGINAL INTERVIEWER FOLLOW-UP

Where would you enforce an external ID for a third-party integration?

Answer: Put the external ID as an sts:ExternalId condition in the target role's trust policy for the third party. Also restrict the trusted principal and grant only the needed operations.

Common mistake: Adding S3 permissions does not fix a role's missing trust relationship.

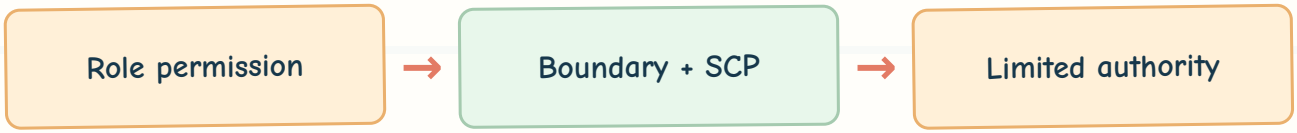
Source guide p. 10 · Further reading: A03-A06

Q009. What do SCPs and permissions boundaries do?

SAY THIS FIRST · SIMPLE ENGLISH

SCPs and permissions boundaries cap possible permissions; they do not grant access by themselves.

Precise interview detail: A service control policy restricts the permissions available to affected principals in organization member accounts. A permissions boundary sets a maximum for an IAM user or role. Neither is an independent permission grant. SCP scope and exceptions, including management-account behavior, matter.



ORIGINAL AWS SCENARIO / COMMAND

```
Role policy: allow a deployment action
Boundary: exclude IAM privilege escalation
SCP: restrict allowed Regions for member accounts
Effective access: evaluate grants and restrictions
```

WHAT HAPPENS · Use organization controls for guardrails and role policies for operational permissions.

ORIGINAL INTERVIEWER FOLLOW-UP

How would you safely delegate role creation to a deployment team?

Answer: Let deployment teams create only roles with a required permissions boundary. Limit iam:PassRole, role paths, and policy changes; test that created roles cannot escape the guardrails.

Common mistake: 'AdministratorAccess' does not bypass an applicable organizational explicit deny.

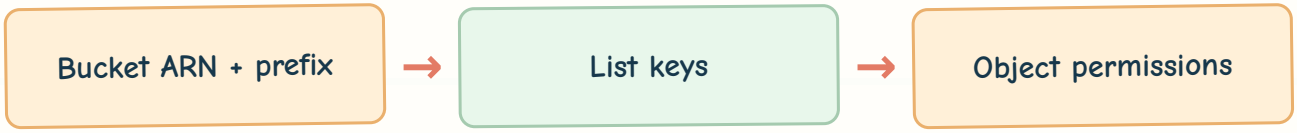
Source guide p. 11 · Further reading: A03-A06

Q010. What is least privilege for an application?

SAY THIS FIRST · SIMPLE ENGLISH

Least privilege means giving an application only the actions and resource paths it actually needs.

Precise interview detail: Grant the smallest practical actions and resource scope that satisfy observed business operations. Use conditions where supported and separate unrelated workloads. Review real access, test denied paths and revise policies when the workload changes.



ORIGINAL AWS SCENARIO / COMMAND

```
{
 "Effect":"Allow", "Action":"s3:GetObject",
 "Resource":"arn:aws:s3:::EXAMPLE_BUCKET/reports/*"
}
```

WHAT HAPPENS · This statement covers object reads under one prefix, not bucket listing or KMS decryption.

ORIGINAL INTERVIEWER FOLLOW-UP

How would you add prefix-restricted ListBucket access?

Answer: Add a separate s3:ListBucket statement against the bucket ARN and restrict s3:prefix with a condition such as reports/*. Object reads still need a separate object ARN statement.

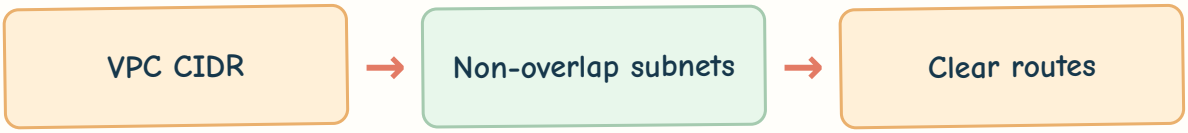
Common mistake: An action can require a bucket ARN, object ARN, other service permission or a resource policy; ARN shape is not interchangeable.

Source guide p. 11 · Further reading: A03-A06

CHAPTER 03 / 20

VPC and network controls

Trace packets through routes and policy.



YOUR FIVE INTERVIEW QUESTIONS

011. What are a VPC, subnet and route table?

012. What makes a subnet public or private?

013. How do security groups differ from network ACLs?

014. When should you use VPC endpoints instead of NAT?

015. What are VPC peering, Transit Gateway, VPN and Direct Connect?

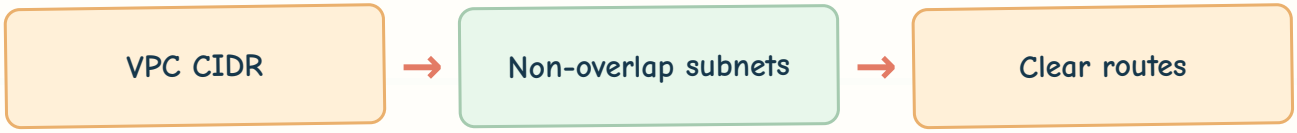
> Say the guarantee and its limit before describing any advanced service.

Q011. What are a VPC, subnet and route table?

SAY THIS FIRST · SIMPLE ENGLISH

A VPC is your AWS network, subnets are AZ-specific IP ranges, and route tables decide where packets should go.

Precise interview detail: A VPC is an isolated network boundary. A subnet occupies one AZ and defines an address range. A route table chooses destinations for traffic leaving the subnet. Route reachability and security rules are separate concerns.



ORIGINAL AWS SCENARIO / COMMAND

```
VPC: 10.20.0.0/16
AZ A app subnet: 10.20.10.0/24
AZ B app subnet: 10.20.20.0/24
Route: destination CIDR -> target
```

WHAT HAPPENS · A multi-AZ service uses subnets in multiple AZs within the intended network.

ORIGINAL INTERVIEWER FOLLOW-UP

Why must subnet address ranges avoid overlap?

Answer: Overlapping addresses make routing ambiguous and complicate VPC peering, Transit Gateway, VPN, or on-prem connectivity. Plan CIDR ranges before connecting networks.

Common mistake: A route allowing reachability does not authorize an application connection.

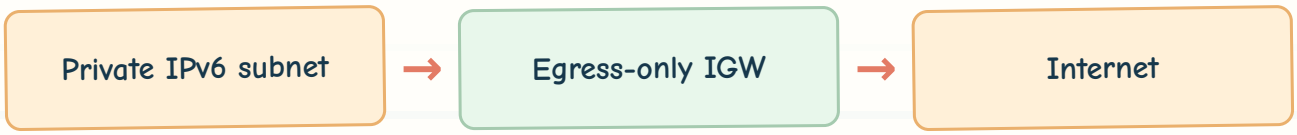
Source guide p. 12 · Further reading: A07-A10, A53, A54

Q012. What makes a subnet public or private?

SAY THIS FIRST · SIMPLE ENGLISH

A public subnet routes toward an internet gateway. A private subnet does not provide that direct route and can use NAT for outbound IPv4.

Precise interview detail: A public subnet has a route to an internet gateway. Direct IPv4 internet communication also requires an appropriate public address and permissive controls. A private subnet lacks that direct internet-gateway route; it may use a NAT gateway for outbound IPv4 access.



ORIGINAL AWS SCENARIO / COMMAND

```
Public route: 0.0.0.0/0 -> internet gateway
Private route: 0.0.0.0/0 -> NAT gateway
Private task: private IP, no public IP
Database: private subnet, bounded inbound rule
```

WHAT HAPPENS · A task in a private subnet can call public APIs through an established outbound path without accepting unsolicited inbound connections through that NAT.

ORIGINAL INTERVIEWER FOLLOW-UP

How would you handle IPv6 egress differently?

Answer: For private IPv6 outbound traffic, evaluate an egress-only internet gateway and correct IPv6 routes and security controls. NAT Gateway is primarily an IPv4 solution; verify the specific setup.

Common mistake: Assigning a public IP alone does not create a working internet route.

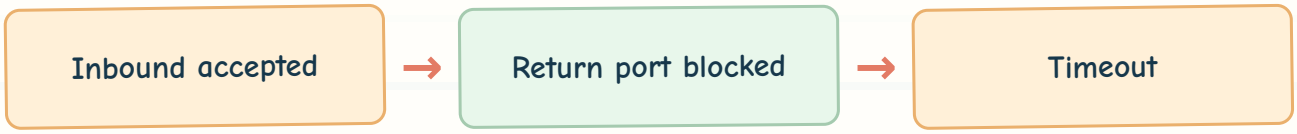
Source guide p. 13 · Further reading: A07-A10, A53, A54

Q013. How do security groups differ from network ACLs?

SAY THIS FIRST · SIMPLE ENGLISH

Security groups are stateful allow rules around resources. Network ACLs are stateless ordered subnet rules that can also deny traffic.

Precise interview detail: Security groups are stateful allow-rule controls on supported resources or network interfaces. Network ACLs are stateless subnet controls with ordered allow and deny rules. Stateless controls need explicit allowance for both directions, including return traffic ports.



ORIGINAL AWS SCENARIO / COMMAND

```
ALB SG: inbound HTTPS from intended clients
Task SG: inbound 8080 from ALB SG
DB SG: inbound 5432 from task SG
NACL: evaluate inbound and outbound separately
```

WHAT HAPPENS · Referencing another security group expresses the permitted traffic source for supported networking arrangements.

ORIGINAL INTERVIEWER FOLLOW-UP

Why can an allowed inbound connection still fail with a restrictive NACL?

Answer: The inbound NACL rule may permit the request while an outbound rule blocks the response's ephemeral port, or vice versa. Inspect both directions and rule order.

Common mistake: A security-group reference does not copy the referenced group's rules.

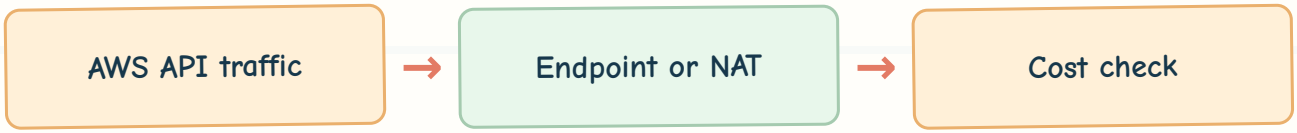
Source guide p. 13 · Further reading: A07-A10, A53, A54

Q014. When should you use VPC endpoints instead of NAT?

SAY THIS FIRST · SIMPLE ENGLISH

Use VPC endpoints for private access to supported AWS services. NAT supports general outbound internet access.

Precise interview detail: Use endpoints when private access to a supported service meets the routing, security and cost requirements. Gateway endpoints are available for S3 and DynamoDB; interface endpoints use private network interfaces for supported services. NAT supports broader outbound destinations.



ORIGINAL AWS SCENARIO / COMMAND

```
S3 traffic -> gateway endpoint route
Secrets Manager -> supported interface endpoint
External payment API -> outbound internet path
Endpoint policy + IAM + resource policy -> evaluate
```

WHAT HAPPENS · An endpoint reduces reliance on public service paths but does not automatically grant service access.

ORIGINAL INTERVIEWER FOLLOW-UP

How would you compare endpoint and NAT costs for your traffic?

Answer: Estimate traffic volume, number of AZs, endpoint hourly charges, data-processing charges, NAT transfer charges, and operational needs. Gateway endpoints for S3/DynamoDB may have different pricing from interface endpoints.

Common mistake: Interface endpoint security groups, DNS settings and endpoint policies can still block requests.

Source guide p. 14 · Further reading: A07-A10, A53, A54

Q015. What are VPC peering, Transit Gateway, VPN and Direct Connect?

SAY THIS FIRST · SIMPLE ENGLISH

Peering connects VPCs, Transit Gateway provides a routing hub, VPN uses encrypted tunnels, and Direct Connect gives dedicated connectivity.

Precise interview detail: Peering links VPCs without transitive routing. Transit Gateway provides a managed hub for multiple networks. Site-to-Site VPN uses encrypted tunnels over network paths. Direct Connect supplies dedicated connectivity, with encryption requiring an explicit design.



ORIGINAL AWS SCENARIO / COMMAND

```
Two VPCs: peering may be enough
Many VPCs + on premises: evaluate a routing hub
VPN: encrypted tunnel
Direct Connect: dedicated connectivity path
```

WHAT HAPPENS · Select connectivity from scale, throughput, latency, routing and resilience needs.

ORIGINAL INTERVIEWER FOLLOW-UP

What redundancy would you add to hybrid connectivity?

Answer: Use redundant customer gateways, VPN tunnels or Direct Connect links in separate locations, diverse devices and paths, monitored BGP failover, and a tested recovery path.

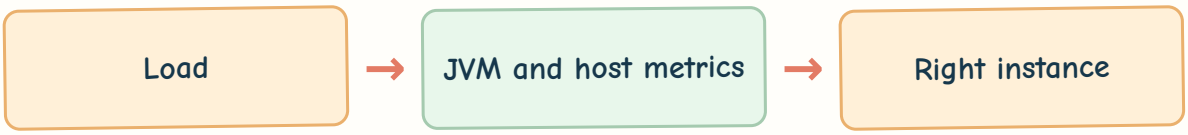
Common mistake: Direct Connect alone should not be described as automatically encrypted or a complete resilient network.

Source guide p. 14 · Further reading: A07-A10, A53, A54

CHAPTER 04 / 20

EC2, Auto Scaling and instance operations

Scale useful work without hiding shared bottlenecks.



YOUR FIVE INTERVIEW QUESTIONS

016. What is EC2 and how do you choose an instance?

017. What happens when an EC2 instance stops or terminates?

018. What is an Auto Scaling group?

019. What is the difference between On-Demand, Spot and commitment discounts?

020. How do you operate EC2 securely without exposing SSH?

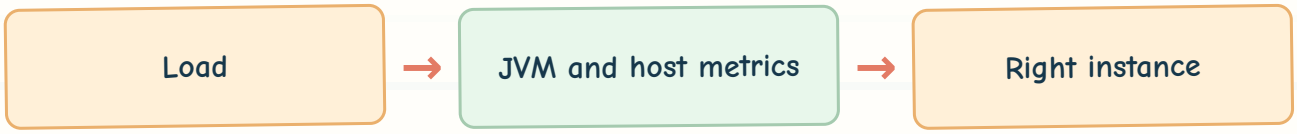
> Say the guarantee and its limit before describing any advanced service.

Q016. What is EC2 and how do you choose an instance?

SAY THIS FIRST · SIMPLE ENGLISH

EC2 rents virtual machines. Choose the CPU, memory, networking, storage, and processor type that match your measured Java workload.

Precise interview detail: EC2 provides virtual compute instances. Select an instance family and size using CPU, memory, network, storage and architecture needs, then measure the workload. Java heap alone is not the whole memory budget.



ORIGINAL AWS SCENARIO / COMMAND

```
Task memory estimate:
heap + metaspace + thread stacks + native buffers
Measure: CPU, allocation rate, GC, RSS, throughput
Check: x86_64 or arm64 dependencies
```

WHAT HAPPENS · A larger machine may postpone a bottleneck without fixing database contention or poor queries.

ORIGINAL INTERVIEWER FOLLOW-UP

How would you benchmark a memory-heavy Java service?

Answer: Load-test with realistic payloads and concurrency; measure RSS, heap, GC pauses, native memory, thread count, p95/p99 latency, CPU, and throughput. Check behavior near the memory ceiling.

Common mistake: Assuming equal vCPU counts produce identical performance ignores generation and workload differences.

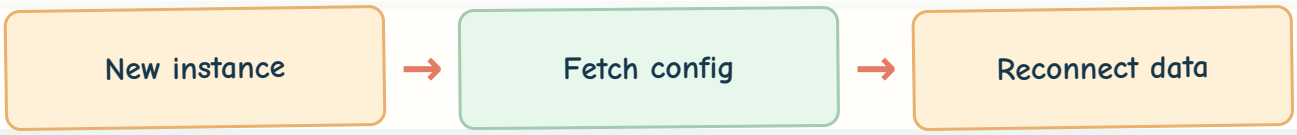
Source guide p. 15 · Further reading: A11-A13, A62

Q017. What happens when an EC2 instance stops or terminates?

SAY THIS FIRST · SIMPLE ENGLISH

Stopping EC2 ends compute but can keep EBS volumes. Termination may remove volumes depending on their settings; instance-store files are temporary.

Precise interview detail: Stopping a supported EBS-backed instance stops its compute execution while preserving attached EBS data. Termination ends the instance and removes volumes configured for deletion. Instance-store data has a temporary lifecycle and should not be treated as durable application storage.



ORIGINAL AWS SCENARIO / COMMAND

```
Instance stopped: process gone, EBS retained
Instance terminated: inspect DeleteOnTermination
Instance store: do not depend on persistent recovery
External data: database or object storage
```

WHAT HAPPENS · Storage retention depends on the storage type and attachment configuration.

ORIGINAL INTERVIEWER FOLLOW-UP

How would you recover a replacement service without local mutable state?

Answer: Build replacement machines from versioned images and infrastructure code. Keep configuration and secrets external and business state in RDS, S3, or durable services; run a failover test.

Common mistake: Stopped instances can still incur charges for retained storage and other resources.

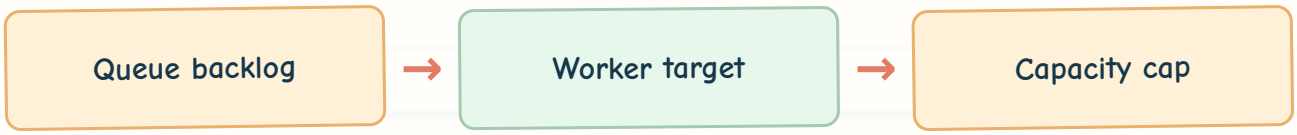
Source guide p. 15 · Further reading: A11-A13, A62

Q018. What is an Auto Scaling group?

SAY THIS FIRST · SIMPLE ENGLISH

An Auto Scaling group keeps the requested number of EC2 instances and can raise or lower capacity using policies.

Precise interview detail: An Auto Scaling group maintains desired instance capacity across configured locations and can adjust it using policies. Launch templates define instance configuration. Health assessment and replacement help restore compute capacity but do not prove that application dependencies are healthy.



ORIGINAL AWS SCENARIO / COMMAND

```
Minimum: 2 instances
Desired: 3 instances
Maximum: 8 instances
Metric: suitable demand per serving instance
```

WHAT HAPPENS · Scaling policies need capacity headroom, initialization time and sensible signals.

ORIGINAL INTERVIEWER FOLLOW-UP

How would you scale a queue worker using backlog per instance?

Answer: Estimate acceptable queue wait and processing time, then scale from backlog per healthy worker or oldest-message age. Add limits for worker startup and database capacity.

Common mistake: Scaling CPU-bound workers using request count alone may miss uneven processing cost.

Source guide p. 16 · Further reading: A11-A13, A62

Q019. What is the difference between On-Demand, Spot and commitment discounts?

SAY THIS FIRST · SIMPLE ENGLISH

On-Demand gives flexible capacity, Spot is cheaper but interruptible, and commitments discount eligible long-term usage.

Precise interview detail: On-Demand suits flexible capacity without a long-term usage commitment. Spot uses spare capacity with interruption risk. Savings Plans and Reserved Instances are discount mechanisms with specific terms and coverage. Capacity reservations are a separate concern and must be evaluated explicitly.



ORIGINAL AWS SCENARIO / COMMAND

```
API baseline: availability-sensitive capacity
Batch overflow: interruption-tolerant Spot workers
Stable measured usage: evaluate commitment coverage
Every option: compare utilization and constraints
```

WHAT HAPPENS · Design Spot workloads to checkpoint, retry and move work when interrupted.

ORIGINAL INTERVIEWER FOLLOW-UP

Which parts of a payment service would you place on Spot?

Answer: Put no irreplaceable payment decision on Spot alone. Spot can host replayable, idempotent reconciliation or analytics workers with checkpoints and durable queues.

Common mistake: A discount commitment should not be described as universally guaranteeing capacity.

Source guide p. 16 · Further reading: A11-A13, A62

Q020. How do you operate EC2 securely without exposing SSH?

SAY THIS FIRST · SIMPLE ENGLISH

Use controlled management access, instance roles, patching, logs, and IMDSv2 rather than keeping SSH open to the world.

Precise interview detail: Use Systems Manager Session Manager where its prerequisites and policies are satisfied, restrict administrative access, use instance roles and protect metadata access. Maintain patching and auditability. Enforce IMDSv2 for compatible workloads and avoid passing instance credentials to unrelated processes.



ORIGINAL AWS SCENARIO / COMMAND

```
Operator -> authorized management session
Instance role -> required service actions
Metadata access -> controlled and IMDSv2-aware
Logs -> centralized audit and diagnostics
```

WHAT HAPPENS · Removing inbound SSH simplifies one access path, but management agents, service reachability and operator permissions still need configuration.

ORIGINAL INTERVIEWER FOLLOW-UP

How would you investigate an instance that cannot establish its management connection?

Answer: Check instance SSM agent status, IAM instance profile, Systems Manager permissions, VPC endpoints or internet path, DNS, proxy, and regional agent support. Inspect managed-node connection status.

Common mistake: IMDSv2 does not replace IAM least privilege or workload isolation.

Source guide p. 17 · Further reading: A11-A13, A62

CHAPTER 05 / 20

S3 and storage choices

Match the application access pattern to the storage service.



YOUR FIVE INTERVIEW QUESTIONS

021. What is the difference between S3, EBS and EFS?

022. Is S3 strongly consistent?

023. What do S3 versioning and lifecycle rules provide?

024. How do presigned URLs work?

025. How do you secure an S3 bucket?

> Say the guarantee and its limit before describing any advanced service.

Q021. What is the difference between S3, EBS and EFS?

SAY THIS FIRST · SIMPLE ENGLISH

S3 is object storage, EBS is a disk-like block volume, and EFS is a shared network filesystem.

Precise interview detail: S3 stores objects accessed through APIs. EBS supplies block storage associated with EC2 availability-zone placement. EFS provides shared file storage using NFS with deployment options and mount targets. Choose from access semantics, sharing, latency and failure requirements.



ORIGINAL AWS SCENARIO / COMMAND

```
Reports and images -> S3 objects
Database disk on EC2 -> EBS blocks
Shared POSIX-style files -> evaluate EFS
Durability requirement -> include backup and restore
```

WHAT HAPPENS · Replacing a filesystem with S3 normally changes the application interface.

ORIGINAL INTERVIEWER FOLLOW-UP

Which choice would you make for uploaded documents in a replicated API?

Answer: Store uploaded documents in S3 and put ownership, upload status, and object keys in a database. Let multiple API replicas read through scoped permissions or presigned URLs.

Common mistake: S3 key prefixes resemble directories but are not general filesystem directories.

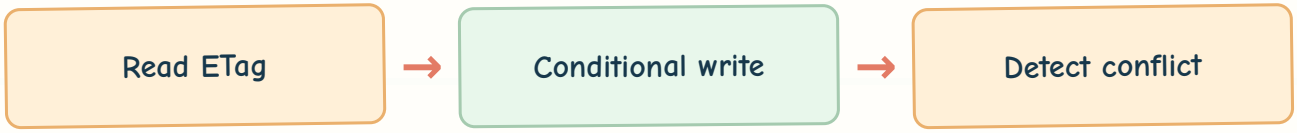
Source guide p. 18 · Further reading: A14-A16, A37

Q022. Is S3 strongly consistent?

SAY THIS FIRST · SIMPLE ENGLISH

S3 has strong consistency for supported object reads and listings after writes, but it does not provide a database transaction across objects.

Precise interview detail: S3 provides strong read-after-write consistency for object writes, overwrites and deletes, including listings. This does not create multi-object transactions or eliminate independent caches, replication delays and application races.



ORIGINAL AWS SCENARIO / COMMAND

```
PUT invoice-42.json succeeds
GET same key sees the successful write
Two writers overwrite one key: app needs coordination
CDN copy: governed by its separate cache behavior
```

WHAT HAPPENS · A client can observe a completed object write without using an old eventual-consistency workaround.

ORIGINAL INTERVIEWER FOLLOW-UP

How would conditional requests protect an update from a lost write?

Answer: Use an ETag-based condition such as If-Match when updating an object so a stale writer fails instead of overwriting a newer version. Handle conflicts explicitly.

Common mistake: Strong object consistency does not make a read-modify-write sequence atomic.

Source guide p. 19 · Further reading: A14-A16, A37

Q023. What do S3 versioning and lifecycle rules provide?

SAY THIS FIRST · SIMPLE ENGLISH

S3 versioning saves separate object versions; lifecycle rules control when objects or versions move or expire.

Precise interview detail: Versioning preserves distinct versions and changes deletion behavior. Lifecycle rules transition or expire eligible objects and versions according to a policy. Together they support retention management, but protection from deletion needs explicit permissions and possibly Object Lock controls.



ORIGINAL AWS SCENARIO / COMMAND

```
Version 1: original invoice
Version 2: corrected invoice
Delete marker: current key appears deleted
Recovery: choose a retained prior version
```

WHAT HAPPENS · Version history helps recover overwritten data when the prior version remains available.

ORIGINAL INTERVIEWER FOLLOW-UP

How would you test a retention policy before enabling it broadly?

Answer: Apply the rule to a test bucket with test versions, preview the selected objects, verify Object Lock and retention expectations, and confirm restore before changing production rules.

Common mistake: Expiring noncurrent versions or broad delete permissions can defeat your recovery plan.

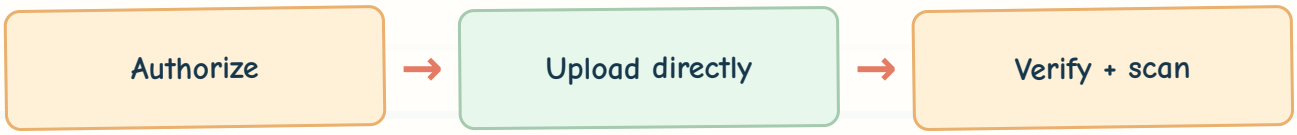
Source guide p. 19 · Further reading: A14-A16, A37

Q024. How do presigned URLs work?

SAY THIS FIRST · SIMPLE ENGLISH

A presigned URL grants temporary permission for a specific S3 operation; anyone holding it can use it while valid.

Precise interview detail: A presigned URL authorizes a specific supported operation using the signer's credentials and signature conditions for a limited validity period. It acts as a bearer capability. Actual usability can end sooner if credentials expire or applicable permissions change.



ORIGINAL AWS SCENARIO / COMMAND

```
API validates customer ownership
API presigns PUT for uploads/customer-42/object-id
Client uploads directly to S3
API verifies the completed object before publishing
```

WHAT HAPPENS · Direct upload can reduce traffic through your Java application.

ORIGINAL INTERVIEWER FOLLOW-UP

How would you constrain upload size, type and downstream trust?

Answer: Authorize the caller first, use a narrow key, short expiry and signed conditions where supported. Verify the stored object's actual size and magic bytes, scan it, and publish only after validation.

Common mistake: A presigned URL can be reused during its usable period; it is not inherently a one-time token.

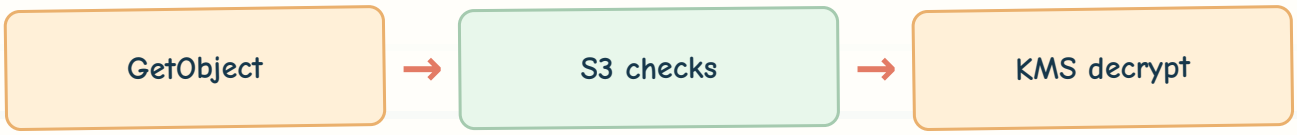
Source guide p. 20 · Further reading: A14-A16, A37

Q025. How do you secure an S3 bucket?

SAY THIS FIRST · SIMPLE ENGLISH

Keep buckets private, scope IAM access, encrypt objects, log sensitive operations, and plan retention and recovery.

Precise interview detail: Use bounded IAM and bucket policies, appropriate Block Public Access settings, encryption, audit signals and retention controls. Prefer explicit ownership and access policies over unnecessary ACLs. Validate both intended access and unintended public or cross-account paths.



ORIGINAL AWS SCENARIO / COMMAND

```
Private bucket -> no public grants
Object prefix -> tenant-aware authorization
SSE-KMS -> evaluate S3 and KMS permissions
Audit -> record relevant data access events as needed
```

WHAT HAPPENS · Encryption at rest protects one threat surface; authorization decides who may retrieve plaintext.

ORIGINAL INTERVIEWER FOLLOW-UP

What would you inspect after an unexpected AccessDenied on an encrypted object?

Answer: Check the real principal, bucket and object ARN, bucket policy, SCP, VPC endpoint policy, key ownership, and KMS key policy or grants. SSE-KMS often needs kms:Decrypt too.

Common mistake: Blocking public access is not a complete defense against an overprivileged internal principal.

Source guide p. 20 · Further reading: A14-A16, A37

CHAPTER 06 / 20

Load balancing, DNS and edge delivery

Separate HTTP routing, DNS and security.



YOUR FIVE INTERVIEW QUESTIONS

026. What is the difference between ALB and NLB?

027. What do load-balancer health checks prove?

028. What does Route 53 do?

029. What is CloudFront and what can be cached?

030. How do WAF, Shield and TLS differ?

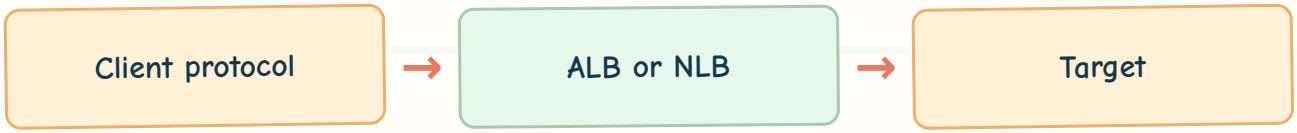
> Say the guarantee and its limit before describing any advanced service.

Q026. What is the difference between ALB and NLB?

SAY THIS FIRST · SIMPLE ENGLISH

ALB understands HTTP paths and hosts. NLB routes lower-level TCP/UDP/TLS traffic.

Precise interview detail: An Application Load Balancer routes HTTP and HTTPS traffic using application-level rules such as paths and hosts. A Network Load Balancer handles transport-level traffic such as TCP and UDP and TLS listeners. Choose from protocol behavior, addressing, traffic scale and target needs.



ORIGINAL AWS SCENARIO / COMMAND

```
ALB: /orders -> order target group
ALB: /catalog -> catalog target group
NLB: TCP traffic -> transport-level targets
TLS termination: choose listener and trust boundary
```

WHAT HAPPENS · For a typical Spring REST API, ALB routing and health checks are often a useful fit.

ORIGINAL INTERVIEWER FOLLOW-UP

When would a non-HTTP protocol push you toward NLB?

Answer: Choose NLB for supported raw TCP or UDP protocols, certain fixed-address needs, or applications where HTTP-aware routing is not appropriate. Check protocol and target requirements.

Common mistake: A load balancer does not repair an unhealthy application or add database capacity.

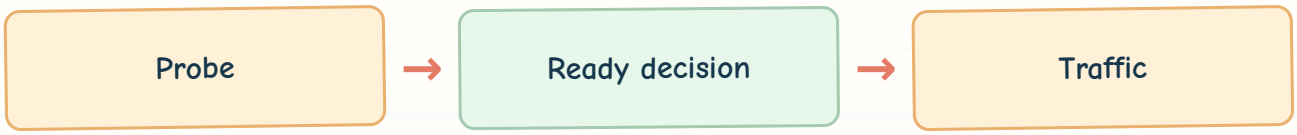
Source guide p. 21 · Further reading: A17, A18, A55, A56, A64, A65

Q027. What do load-balancer health checks prove?

SAY THIS FIRST · SIMPLE ENGLISH

A health check tests a configured endpoint, not every user action or database invariant.

Precise interview detail: They determine target health according to configured requests, thresholds and timings. Design a fast endpoint that reflects traffic readiness. Separate startup and readiness from deep diagnostics so a dependency incident does not cause harmful replacement or admission behavior.



ORIGINAL AWS SCENARIO / COMMAND

```
GET /actuator/health/readiness -> serving readiness
Startup incomplete -> return non-ready status
Dependency incident -> controlled degradation policy
Shutdown -> stop new traffic, drain existing work
```

WHAT HAPPENS · Probe design affects routing and recovery. ALB can fail open when all registered targets are unhealthy, so health status alone is not a hard security or traffic-isolation boundary.

ORIGINAL INTERVIEWER FOLLOW-UP

What should readiness return during a brief database failover?

Answer: Choose readiness based on whether the instance can serve useful requests. During brief failover it may temporarily return unavailable; avoid restarting healthy instances and test the load balancer's all-unhealthy behavior.

Common mistake: A health check that merely reports process existence misses deadlocked request handling.

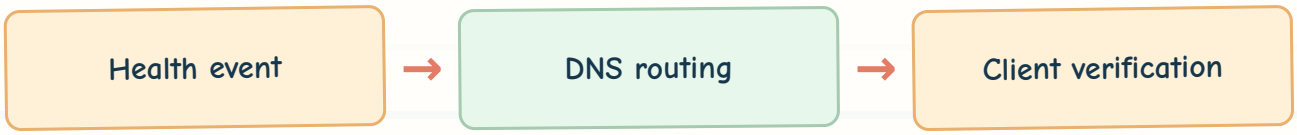
Source guide p. 21 · Further reading: A17, A18, A55, A56, A64, A65

Q028. What does Route 53 do?

SAY THIS FIRST · SIMPLE ENGLISH

Route 53 turns names into addresses and supports different routing policies. DNS caches can delay failover.

Precise interview detail: Route 53 provides DNS functionality and routing policies, with associated health-check capabilities. DNS resolves names; clients and resolvers cache responses. DNS failover therefore depends on health detection, TTLs, caches and client behavior rather than an instantaneous global switch.



ORIGINAL AWS SCENARIO / COMMAND

```
api.example.com -> intended service endpoint
Policy -> simple, weighted, latency or failover design
TTL -> cache guidance for recursive resolvers
Client pool -> may retain existing connections
```

WHAT HAPPENS · Plan failover for new resolution and existing connections.

ORIGINAL INTERVIEWER FOLLOW-UP

How would you verify a regional failover from multiple client networks?

Answer: Test from different networks and Regions, resolve the name repeatedly, compare actual endpoint IPs, send application requests, and check recovery time for both new and existing connections.

Common mistake: Lower TTL does not force every client to reconnect immediately.

Source guide p. 22 · Further reading: A17, A18, A55, A56, A64, A65

Q029. What is CloudFront and what can be cached?

SAY THIS FIRST · SIMPLE ENGLISH

CloudFront caches eligible content near users. Cache keys and privacy settings decide which response can be reused.

Precise interview detail: CloudFront delivers content through edge locations and can cache eligible responses from origins. Configure cache keys and policies around the response's true variation: path, selected headers, query parameters and cookies. Authenticated personalized content needs particular care.



ORIGINAL AWS SCENARIO / COMMAND

```
Static /assets/app-HASH.js -> long immutable cache
Product page -> bounded TTL and correct key
Personal invoice -> explicit protected delivery policy
Invalidation or new object key -> update strategy
```

WHAT HAPPENS · A cache hit avoids some origin work but may serve an older permitted representation.

ORIGINAL INTERVIEWER FOLLOW-UP

When would versioned object names be preferable to invalidation?

Answer: Use content-hashed file names for static JavaScript, images, and assets so old and new releases can coexist and long cache lifetimes are safe. Invalidate when names cannot change.

Common mistake: Omitting a tenant-dependent header from a shared cache key can expose one tenant's response to another.

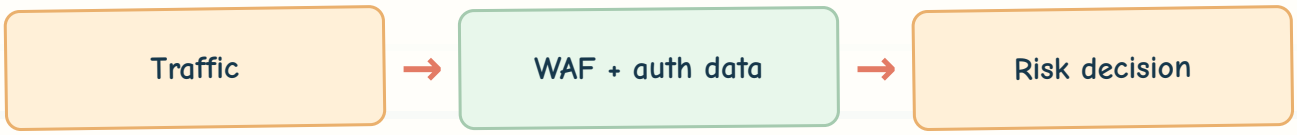
Source guide p. 22 · Further reading: A17, A18, A55, A56, A64, A65

Q030. How do WAF, Shield and TLS differ?

SAY THIS FIRST · SIMPLE ENGLISH

TLS encrypts transport, WAF filters selected web traffic, and Shield helps defend against DDoS.

Precise interview detail: TLS protects data in transit and authenticates endpoints according to certificate validation. AWS WAF evaluates web requests against rules on supported integrations. AWS Shield addresses DDoS protection with different service tiers. These solve overlapping operational concerns but do not replace application authorization.



ORIGINAL AWS SCENARIO / COMMAND

```
TLS -> secure connection to intended endpoint
WAF -> reject selected suspicious HTTP patterns
App -> authenticate user and authorize invoice access
Rate policy -> protect bounded backend capacity
```

WHAT HAPPENS · Layer edge controls with correct identity and resource ownership checks.

ORIGINAL INTERVIEWER FOLLOW-UP

How would you distinguish credential stuffing from a large legitimate sale?

Answer: Compare account-level failed logins, IP/device patterns, bot indicators, conversion rates, and authenticated activity. Apply user-aware limits without blocking legitimate demand only because traffic is high.

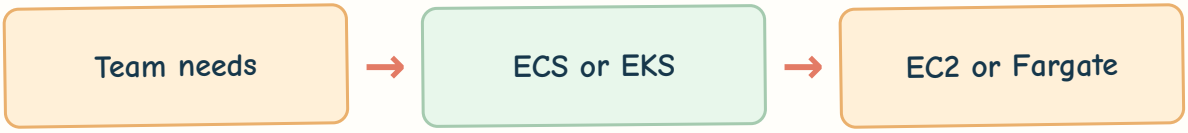
Common mistake: A WAF rule cannot guarantee that every request is safe or that business-level abuse is prevented.

Source guide p. 23 · Further reading: A17, A18, A55, A56, A64, A65

CHAPTER 07 / 20

ECS, EKS and Java containers

Choose an orchestrator and compute model separately.



YOUR FIVE INTERVIEW QUESTIONS

031. How do ECS, EKS and Fargate differ?

032. What are ECS task definitions, tasks and services?

033. How do the ECS task role and execution role differ?

034. How do you make Spring Boot ready for container deployment?

035. Who is responsible for security in EKS?

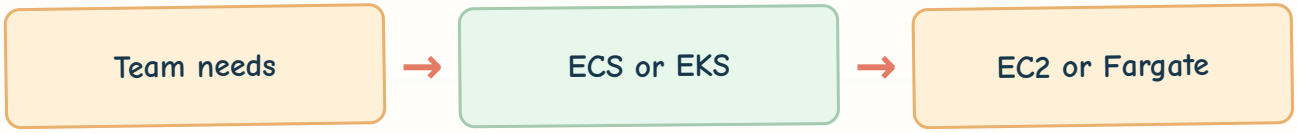
> Say the guarantee and its limit before describing any advanced service.

Q031. How do ECS, EKS and Fargate differ?

SAY THIS FIRST · SIMPLE ENGLISH

ECS runs AWS-managed container workloads. EKS runs Kubernetes workloads. Fargate is a compute choice available for supported workloads.

Precise interview detail: ECS is AWS container orchestration. EKS provides managed Kubernetes control-plane capabilities. Fargate supplies a managed compute option for supported ECS and EKS workloads. Orchestration choice and compute choice are separate decisions.



ORIGINAL AWS SCENARIO / COMMAND

```
ECS + Fargate -> managed task scheduling and compute
ECS + EC2 -> customer-managed container host capacity
EKS -> Kubernetes APIs and ecosystem
Choose -> skills, platform needs, operations and cost
```

WHAT HAPPENS · Fargate removes much host administration while leaving task configuration, permissions and application operation with you.

ORIGINAL INTERVIEWER FOLLOW-UP

What would justify EKS for a team already productive on ECS?

Answer: Choose EKS when standard Kubernetes APIs, operators, custom scheduling, or an established K8s platform justify its extra operational work; not just because it sounds more advanced.

Common mistake: Describing Fargate as a replacement for every Kubernetes capability misses workload restrictions.

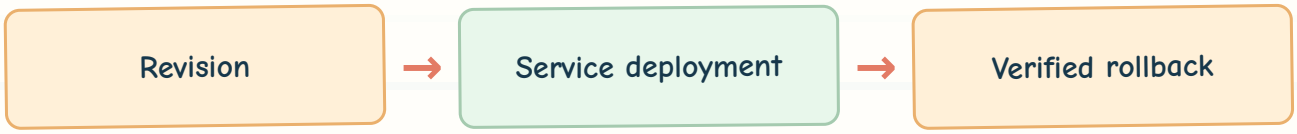
Source guide p. 24 · Further reading: A19-A21, A43, A49, A50

Q032. What are ECS task definitions, tasks and services?

SAY THIS FIRST · SIMPLE ENGLISH

An ECS task definition is a blueprint, a task is one running copy, and a service maintains desired tasks.

Precise interview detail: A task definition describes a workload configuration, including containers and resources. A task is a running instance of that definition. A service maintains desired task count and manages deployment and optional load-balancer integration.



ORIGINAL AWS SCENARIO / COMMAND

```
Task definition revision: orders:17
Task: one launched instance of orders:17
Service desired count: 3
New revision: deployment replaces eligible old tasks
```

WHAT HAPPENS · The service is responsible for ongoing reconciliation rather than a single task launch.

ORIGINAL INTERVIEWER FOLLOW-UP

How would you roll back a failed service deployment?

Answer: Deploy a known-good task-definition revision or enable an appropriate deployment circuit breaker and rollback. Validate database compatibility and user-facing metrics after restoration.

Common mistake: Registering a task definition does not start a service or move production traffic.

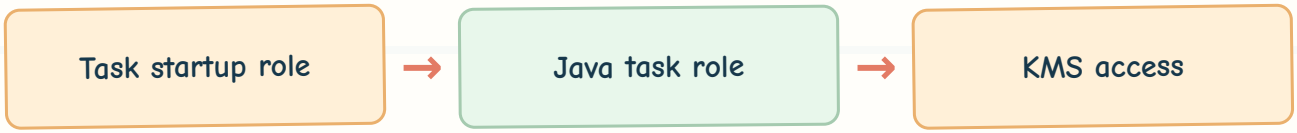
Source guide p. 25 · Further reading: A19-A21, A43, A49, A50

Q033. How do the ECS task role and execution role differ?

SAY THIS FIRST · SIMPLE ENGLISH

The ECS task role is for application AWS calls. The execution role is for supported ECS startup actions such as image pulls and logs.

Precise interview detail: The task role supplies application permissions to AWS APIs. The execution role supplies permissions used by ECS or Fargate agents for supported startup and execution activities, such as pulling images, sending logs and resolving configured secrets. Keep these identities separate.



ORIGINAL AWS SCENARIO / COMMAND

```
taskRoleArn -> application reads reports and sends SQS
executionRoleArn -> image pull and log delivery
secrets injection -> execution-role permission path
SDK secret fetch -> application task-role path
```

WHAT HAPPENS · Who retrieves a value determines which principal needs permission.

ORIGINAL INTERVIEWER FOLLOW-UP

Which role needs KMS access for a secret retrieved by Java at runtime?

Answer: If Java itself calls Secrets Manager and then KMS is needed, the task role and relevant key policy must allow that action. Execution-role access applies to ECS-managed secret injection.

Common mistake: Adding an S3 permission only to the execution role does not grant the application's SDK that permission.

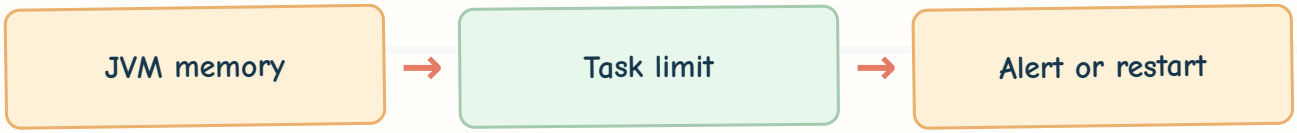
Source guide p. 25 · Further reading: A19-A21, A43, A49, A50

Q034. How do you make Spring Boot ready for container deployment?

SAY THIS FIRST · SIMPLE ENGLISH

Use an immutable Java image, external configuration, limited JVM memory, readiness checks, graceful shutdown, and workload credentials.

Precise interview detail: Build an immutable reviewed image, externalize configuration, budget total JVM memory, expose bounded health endpoints and implement graceful shutdown. Tune startup, request timeouts and connection pools for platform limits. Use workload credentials through the SDK provider chain.



ORIGINAL AWS SCENARIO / COMMAND

```
server.shutdown=graceful
spring.lifecycle.timeout-per-shutdown-phase=20s
management.endpoint.health.probes.enabled=true
# Select exposure and access controls deliberately.
```

WHAT HAPPENS · The platform and application must agree on admission and shutdown timing.

ORIGINAL INTERVIEWER FOLLOW-UP

How would you detect memory pressure before task replacement?

Answer: Watch container RSS and memory utilization, GC activity, native memory and thread count, JVM heap usage, and OOM kills. Alert before approaching task memory limits.

Common mistake: Heap size equal to the container memory limit leaves no room for native allocations and threads.

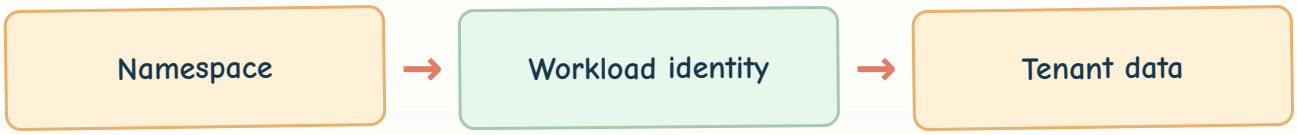
Source guide p. 26 · Further reading: A19-A21, A43, A49, A50

Q035. Who is responsible for security in EKS?

SAY THIS FIRST · SIMPLE ENGLISH

AWS operates the managed EKS control plane, while your team must secure workloads, identities, data, and supported node configuration.

Precise interview detail: AWS manages the EKS control-plane responsibilities defined by the service. Customers manage application code, workload identity and access, policies and data. Responsibilities for nodes and OS maintenance depend on the selected compute option and EKS mode.



ORIGINAL AWS SCENARIO / COMMAND

```
Cluster -> control-plane service boundary
Workloads -> image, RBAC, secrets, network policy
Compute -> EC2 nodes or supported managed option
AWS access -> workload identity, bounded IAM role
```

WHAT HAPPENS · The managed control plane does not make every pod trustworthy or every network policy effective.

ORIGINAL INTERVIEWER FOLLOW-UP

How would you keep one namespace from reading another application's AWS data?

Answer: Give each workload a dedicated service account and scoped IAM role, plus namespace RBAC and network policies. Enforce tenant ownership in the application's data layer.

Common mistake: Giving all pods the node's broad IAM authority destroys workload separation.

Source guide p. 26 · Further reading: A19-A21, A43, A49, A50

CHAPTER 08 / 20

Lambda and API Gateway

Budget concurrency and understand each retry path.



YOUR FIVE INTERVIEW QUESTIONS

036. What is Lambda and when is it a good fit?

037. What is a cold start and how do you reduce its effect for Java?

038. How does Lambda concurrency affect downstream services?

039. How are Lambda failures retried?

040. What does API Gateway add to an API?

> Say the guarantee and its limit before describing any advanced service.

Q036. What is Lambda and when is it a good fit?

SAY THIS FIRST · SIMPLE ENGLISH

Lambda runs short-lived event or API logic without operating servers directly. Compare its execution limits with your workload.

Precise interview detail: Lambda runs functions on managed compute in response to supported invocations. It suits event-driven work and APIs that fit its execution constraints. Compare startup behavior, duration, resource needs, concurrency and integration costs against containers or other compute.



ORIGINAL AWS SCENARIO / COMMAND

```
S3 event -> thumbnail function
Scheduled event -> bounded cleanup job
HTTP integration -> short API operation
Long continuous process -> evaluate other compute
```

WHAT HAPPENS · There is no application server fleet for you to operate, but functions still need resource budgets and deployment discipline.

ORIGINAL INTERVIEWER FOLLOW-UP

What makes a streaming socket service a poor fit for a simple function model?

Answer: A continuously connected socket needs long-lived state, session routing, and connections. Simple Lambda invocation is usually a poor match without an additional managed connection service and state layer.

Common mistake: 'Serverless' does not mean unlimited duration or zero operations.

Source guide p. 27 · Further reading: A22-A24, A56

Q037. What is a cold start and how do you reduce its effect for Java?

SAY THIS FIRST · SIMPLE ENGLISH

A cold start is the extra startup work before Lambda handles a request. Keep Java initialization lean and measure it.

Precise interview detail: A cold start includes creating and initializing an execution environment before useful handling. Reduce avoidable initialization, use suitable packaging and evaluate provisioned concurrency or supported SnapStart configurations. Measure end-to-end latency under actual traffic and deployment patterns.



ORIGINAL AWS SCENARIO / COMMAND

```
Initialization: load classes, construct clients
Invocation: validate input, perform business operation
Warm reuse: retain safe reusable clients
Restore-aware features: validate initialization state
```

WHAT HAPPENS · Sharing SDK clients across invocations can avoid repeated setup. SnapStart needs special care with uniqueness, credentials and restore-safe connections.

ORIGINAL INTERVIEWER FOLLOW-UP

How would you measure cold and warm p99 separately?

Answer: Tag invocation metrics by cold versus warm path using init indicators and tracing, then compare separate p99 distributions under representative traffic. Do not average different p99 values.

Common mistake: Cached mutable request data can leak between invocations in a reused environment.

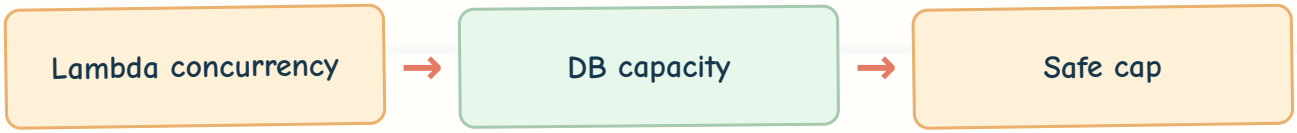
Source guide p. 28 · Further reading: A22-A24, A56

Q038. How does Lambda concurrency affect downstream services?

SAY THIS FIRST · SIMPLE ENGLISH

Lambda can rapidly run many copies, but each copy may put pressure on a limited database or API.

Precise interview detail: Concurrency is the number of simultaneous executions. Reserved concurrency can bound a function's use and reserve part of account capacity; provisioned concurrency prepares execution environments. Downstream connection and rate budgets must fit the admitted concurrency.



ORIGINAL AWS SCENARIO / COMMAND

```
Example: 100 executions x 1 DB connection = 100
Database safe application budget: 60 connections
Controls: cap concurrency, pool/proxy, queue work
Measure: throttles, duration, backlog and DB saturation
```

WHAT HAPPENS · A highly elastic function tier can overwhelm a comparatively fixed database.

ORIGINAL INTERVIEWER FOLLOW-UP

How would you choose a safe concurrency limit during a traffic spike?

Answer: Measure maximum safe database connections and execution time; reserve downstream capacity, set concurrency limits, use queues or RDS Proxy where suitable, then load-test expected bursts.

Common mistake: Provisioned concurrency and reserved concurrency solve different problems.

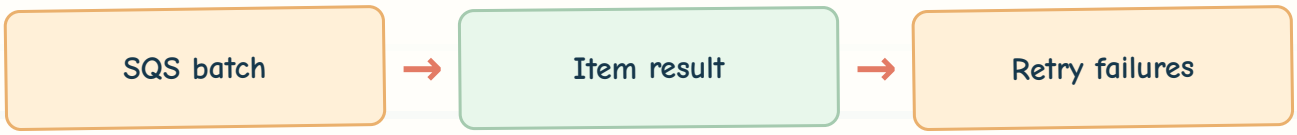
Source guide p. 28 · Further reading: A22-A24, A56

Q039. How are Lambda failures retried?

SAY THIS FIRST · SIMPLE ENGLISH

Lambda retry rules depend on whether calls are synchronous, asynchronous, or read from an event source such as SQS.

Precise interview detail: Retry behavior depends on the invocation path. Synchronous callers own their retry decisions. Asynchronous invocation has service-managed retry and failure-handling settings. Event-source mappings such as SQS have their own processing and retry contracts.



ORIGINAL AWS SCENARIO / COMMAND

```
HTTP caller timeout -> caller decides next action
Async event -> inspect retry and destination settings
SQS mapping -> message visibility and batch handling
Every path -> business operation must tolerate replay
```

WHAT HAPPENS · Configure retry duration and failure destinations for the selected path.

ORIGINAL INTERVIEWER FOLLOW-UP

How would partial batch failure reporting change SQS processing?

Answer: Report only failed item IDs in the batch response when supported, allowing successful items to stay complete. Keep each item's effect idempotent because retries still happen.

Common mistake: One universal 'Lambda retries twice' claim is incorrect across invocation modes.

Source guide p. 29 · Further reading: A22-A24, A56

Q040. What does API Gateway add to an API?

SAY THIS FIRST · SIMPLE ENGLISH

API Gateway provides API ingress features such as routing and authorization, depending on its API type.

Precise interview detail: API Gateway provides managed API ingress capabilities, with features varying between HTTP, REST and WebSocket APIs. It can integrate with functions or other backends and support authentication, routing and traffic controls. Select an API type from required capabilities.



ORIGINAL AWS SCENARIO / COMMAND

```
Client -> API route -> authorizer -> integration
Backend -> status and payload mapping
Controls -> supported throttling and access logs
Application -> resource ownership and invariants
```

WHAT HAPPENS · Gateway authentication establishes who calls; the backend still needs business authorization.

ORIGINAL INTERVIEWER FOLLOW-UP

Why might an ALB be preferable for an existing container REST service?

Answer: An existing ECS REST API may use ALB when straightforward HTTP routing, familiar health checks, and suitable cost/performance are enough. Compare authorization and throttling needs first.

Common mistake: Throttling is not a transaction-level guarantee against abuse or unlimited spend.

Source guide p. 29 · Further reading: A22-A24, A56

CHAPTER 09 / 20

RDS, Aurora and relational reliability

Protect local transactions during failover.



YOUR FIVE INTERVIEW QUESTIONS

041. What do RDS and Aurora manage?

042. How do Multi-AZ and read replicas differ?

043. What happens to a Java application during database failover?

044. How do you size connection pools and use RDS Proxy?

045. How do you handle backups and schema migrations?

> Say the guarantee and its limit before describing any advanced service.

Q041. What do RDS and Aurora manage?

SAY THIS FIRST · SIMPLE ENGLISH

RDS manages supported relational database infrastructure; Aurora is a managed relational family with its own storage architecture.

Precise interview detail: RDS provides managed relational database offerings with engine-specific capabilities. Aurora is a compatible managed database family with its own distributed storage and topology. AWS manages service infrastructure; you still own schema design, SQL performance, access and application transaction behavior.



ORIGINAL AWS SCENARIO / COMMAND

```
Java service -> JDBC -> relational endpoint
Customer: indexes, transactions, migrations, access
Service: managed infrastructure within its contract
Design: availability, backup, read/write requirements
```

WHAT HAPPENS · Choose from engine compatibility, operational requirements, topology and measured workload rather than branding.

ORIGINAL INTERVIEWER FOLLOW-UP

Which parts of a PostgreSQL migration would you verify before choosing Aurora?

Answer: Test PostgreSQL extensions, SQL behavior, version support, transaction isolation, parameter settings, replication, networking, failover, and query plans with production-like data.

Common mistake: Managed databases do not automatically fix inefficient queries or lock contention.

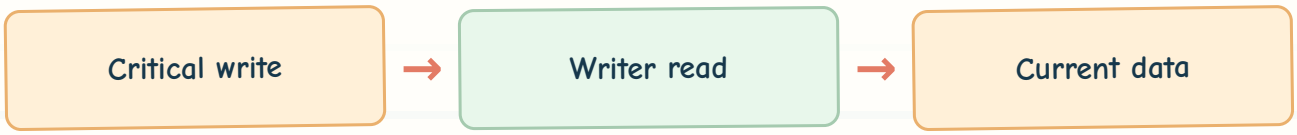
Source guide p. 30 · Further reading: A25-A27, A42

Q042. How do Multi-AZ and read replicas differ?

SAY THIS FIRST · SIMPLE ENGLISH

Multi-AZ is mainly for availability; read replicas primarily add read capacity. Exact RDS topologies have different rules.

Precise interview detail: Multi-AZ deployment options support availability, while read replicas support read scaling and other workflows. A traditional RDS Multi-AZ DB instance uses a standby that does not serve reads. Multi-AZ DB clusters have readable instances; Aurora has its own cluster architecture.



ORIGINAL AWS SCENARIO / COMMAND

```
Traditional Multi-AZ instance -> primary + standby
Read replica -> async read copy, lag may exist
Multi-AZ DB cluster -> different readable topology
Application -> choose writer and reader endpoints
```

WHAT HAPPENS · Always identify the exact RDS deployment type before describing behavior.

ORIGINAL INTERVIEWER FOLLOW-UP

Where should a read immediately after a critical write go?

Answer: Send a critical read-after-write to the writer or another path with a proven fresh-read guarantee. An asynchronous read replica may still contain the old value.

Common mistake: 'Every Multi-AZ standby serves reads' and 'no Multi-AZ instance can ever serve reads' both hide topology distinctions.

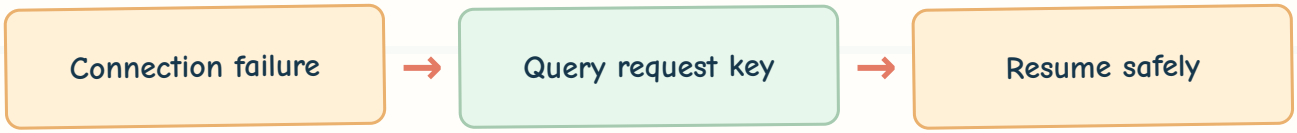
Source guide p. 31 · Further reading: A25-A27, A42

Q043. What happens to a Java application during database failover?

SAY THIS FIRST · SIMPLE ENGLISH

During DB failover, old connections can break and an in-flight commit may have an unknown outcome.

Precise interview detail: Connections may fail, transactions may abort or outcomes may become uncertain. Clients must resolve the appropriate endpoint, refresh invalid connections and retry only operations whose semantics permit it. Failover completion at the service level is not identical to application recovery.



ORIGINAL AWS SCENARIO / COMMAND

```
Connection lost before commit -> likely retryable path
Connection lost during commit -> outcome may be unknown
Client request key -> resolve previous outcome
Connection pool -> evict broken connections
```

WHAT HAPPENS · Use bounded reconnects and idempotency for operations that might have committed.

ORIGINAL INTERVIEWER FOLLOW-UP

How would you determine whether the first transaction committed?

Answer: Query a durable request ID or unique transaction record at the new writer to see if the effect exists. Only retry with the same idempotency identity after checking the outcome.

Common mistake: Retrying a payment after any connection exception can charge twice.

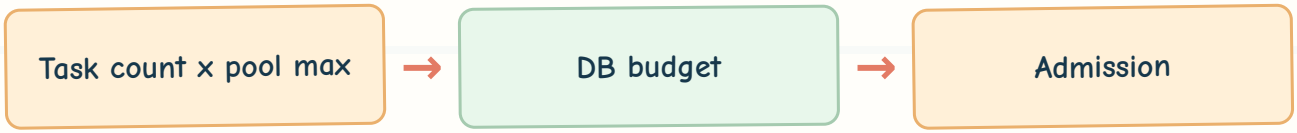
Source guide p. 31 · Further reading: A25-A27, A42

Q044. How do you size connection pools and use RDS Proxy?

SAY THIS FIRST · SIMPLE ENGLISH

The total database connection demand is the sum across all tasks and jobs, not just one HikariCP pool.

Precise interview detail: Budget connections across all replicas and jobs, leaving operational headroom. RDS Proxy can pool and manage connections for supported engines, but session state, pinning and transaction behavior affect benefits. Query optimization and concurrency control remain necessary.



ORIGINAL AWS SCENARIO / COMMAND

```
20 tasks x pool max 15 = up to 300 connections
Database budget for app: 220
Adjust: task count, pool, proxy and query duration
Measure: wait time, active sessions, pinned sessions
```

WHAT HAPPENS · A local pool configuration multiplies across the deployment.

ORIGINAL INTERVIEWER FOLLOW-UP

What happens when autoscaling doubles the number of Java tasks?

Answer: Doubling 20 tasks with pool 15 raises possible connections from 300 to 600. Limit task scaling, reduce pool sizes, and reserve capacity for maintenance; RDS Proxy is not infinite capacity.

Common mistake: A proxy does not increase the database's ability to execute unlimited concurrent work.

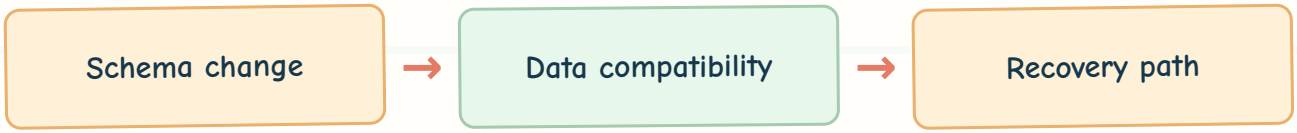
Source guide p. 32 · Further reading: A25-A27, A42

Q045. How do you handle backups and schema migrations?

SAY THIS FIRST · SIMPLE ENGLISH

Backups protect recoverable data; expand-and-contract schema changes let old and new app versions coexist during rollout.

Precise interview detail: Use recovery mechanisms appropriate to the database, retention and independence requirements. Test restores into an isolated environment. Deploy backward-compatible expand-and-contract migrations so old and new application versions can coexist during rollout.



ORIGINAL AWS SCENARIO / COMMAND

```
Expand: add nullable column and compatible readers
Migrate: backfill and verify
Switch: use new column after compatibility checks
Contract: remove old column in a later release
```

WHAT HAPPENS · Rollback must account for data changes, not just application images.

ORIGINAL INTERVIEWER FOLLOW-UP

How would you roll back an application after a destructive migration?

Answer: A destructive migration may make code rollback unsafe. Restore a tested snapshot or use a compatible forward correction and reconciled data plan; define the rollback boundary before deploying.

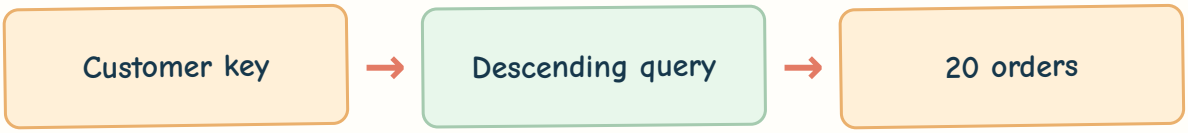
Common mistake: A successful backup job without a restore test does not demonstrate a usable recovery path.

Source guide p. 32 · Further reading: A25-A27, A42

CHAPTER 10 / 20

DynamoDB data modeling

Model keys from queries and traffic distribution.



YOUR FIVE INTERVIEW QUESTIONS

046. How does DynamoDB differ from a relational database?

047. What causes a hot partition or hot key?

048. Which DynamoDB reads can be strongly consistent?

049. How do conditional writes support idempotency?

050. What do DynamoDB TTL and Streams provide?

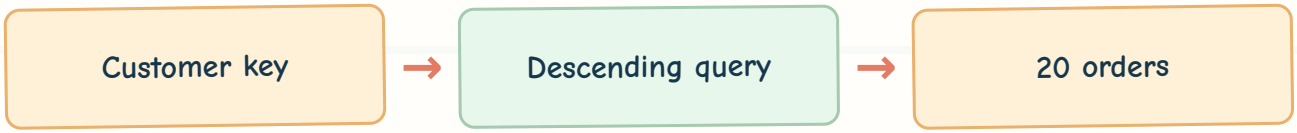
> Say the guarantee and its limit before describing any advanced service.

Q046. How does DynamoDB differ from a relational database?

SAY THIS FIRST · SIMPLE ENGLISH

DynamoDB uses keys and access patterns for fast lookups; relational databases are better for many joined queries and constraints.

Precise interview detail: DynamoDB is a managed key-value and document database built around explicit keys and access patterns. Model efficient queries and distribution before writing application code. It supports conditional operations and transactions, but general joins and arbitrary relational querying are not its core model.



ORIGINAL AWS SCENARIO / COMMAND

```
Access pattern: get one order by customer and ID
PK = CUSTOMER#42
SK = ORDER#2026-10-09#123
Query -> bounded key range, not whole-table scan
```

WHAT HAPPENS · Key design makes the requested operation predictable and bounded.

ORIGINAL INTERVIEWER FOLLOW-UP

How would you list a customer's last 20 orders?

Answer: Partition by customer and sort by order time plus a unique tie-breaker; Query the customer's key in descending order with Limit 20, avoiding a full table scan.

Common mistake: Translating every SQL table directly into a DynamoDB table can miss the workload's actual query needs.

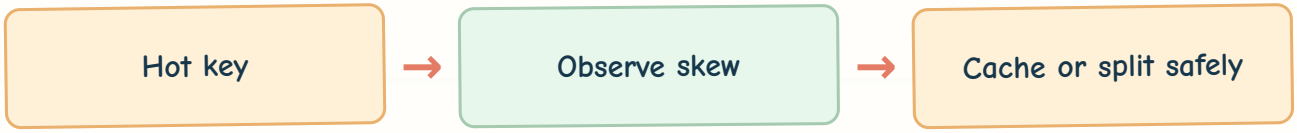
Source guide p. 33 · Further reading: A28-A30, A59, A67

Q047. What causes a hot partition or hot key?

SAY THIS FIRST · SIMPLE ENGLISH

A hot DynamoDB key receives much more traffic than others, even if overall capacity looks fine.

Precise interview detail: Skewed access concentrates work on a limited key space even when aggregate capacity appears sufficient. Choose keys and access patterns that distribute demand. Caching, sharding or write aggregation may help, but each changes application behavior.



ORIGINAL AWS SCENARIO / COMMAND

```
Poor key: PK = STATUS#OPEN for every new order
Better starting point: tenant/customer-based keys
High-volume tenant: evaluate controlled sharding
Query fan-out: account for the new read cost
```

WHAT HAPPENS · Distribution must match real traffic, including a single unusually large customer.

ORIGINAL INTERVIEWER FOLLOW-UP

How would you detect and relieve a single celebrity-item hot key?

Answer: Use per-key metrics and throttling patterns, cache popular reads, and split safe write counters or workloads when business semantics permit. Do not shard exact stock decisions without coordination.

Common mistake: High cardinality alone does not guarantee evenly distributed requests.

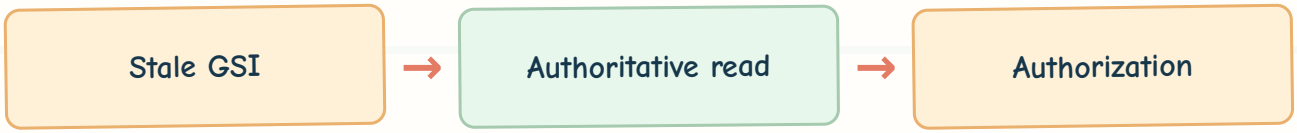
Source guide p. 33 · Further reading: A28-A30, A59, A67

Q048. Which DynamoDB reads can be strongly consistent?

SAY THIS FIRST · SIMPLE ENGLISH

Some DynamoDB table and LSI reads may be strongly consistent; GSI reads are eventually consistent.

Precise interview detail: Tables and local secondary indexes support strongly consistent reads where applicable. Global secondary indexes support eventually consistent reads. Global tables have consistency modes and Region constraints that must be selected and verified; do not assume all multi-Region configurations behave alike.



ORIGINAL AWS SCENARIO / COMMAND

```
Table GetItem: request ConsistentRead where supported
GSI query: eventual consistency
Critical write result: return known committed data
Projection query: tolerate or detect freshness lag
```

WHAT HAPPENS · Select consistency per operation and index, and consider its capacity consequences.

ORIGINAL INTERVIEWER FOLLOW-UP

How would you prevent a stale projection from authorizing an action?

Answer: Do not authorize a sensitive action using a possibly stale GSI alone. Recheck the item or permission against the authoritative table or dedicated trusted store at decision time.

Common mistake: Reading a GSI immediately after writing the base table can return a stale projection.

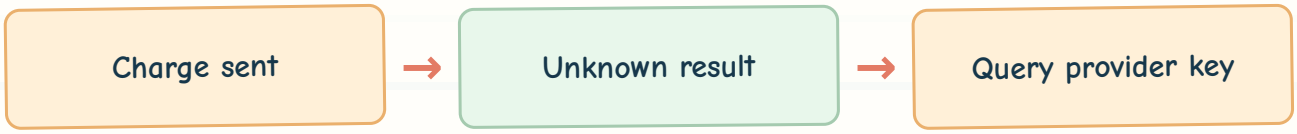
Source guide p. 34 · Further reading: A28-A30, A59, A67

Q049. How do conditional writes support idempotency?

SAY THIS FIRST · SIMPLE ENGLISH

A conditional write accepts a change only if the stored condition is true; this can guard a durable request key.

Precise interview detail: A conditional write applies only when its condition is satisfied. Use a durable request key and a state model to distinguish a first attempt, an in-progress attempt and a completed result. Keep related effects inside an appropriate transaction where supported.



ORIGINAL AWS SCENARIO / COMMAND

```
Key: REQUEST#client-key
Condition: attribute_not_exists(PK)
State: IN_PROGRESS -> COMPLETED with result
Duplicate: read state and return resolved outcome
```

WHAT HAPPENS · The conditional claim must be combined with a recovery policy for abandoned processing.

ORIGINAL INTERVIEWER FOLLOW-UP

How would you recover after a crash between the charge and saving the result?

Answer: Treat charge outcome as unknown, then query the payment provider using the same stable idempotency key or reconcile transaction ID. Store the result durably before responding and never issue an unrelated second charge.

Common mistake: Claiming a key and then charging an external payment provider is not one atomic transaction.

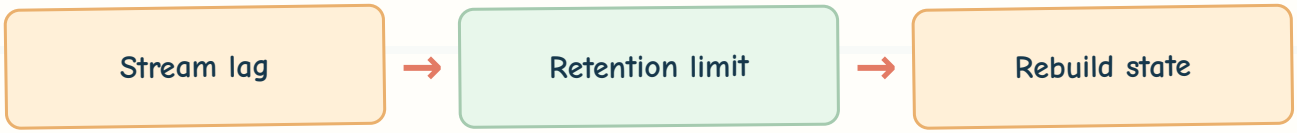
Source guide p. 34 · Further reading: A28-A30, A59, A67

Q050. What do DynamoDB TTL and Streams provide?

SAY THIS FIRST · SIMPLE ENGLISH

DynamoDB TTL expires eligible items asynchronously. Streams exposes ordered item changes within defined scopes and retention limits.

Precise interview detail: TTL enables eventual removal of eligible expired items; it is not a precise deletion timer. Streams capture item-change records for consumers, with defined retention and processing behavior. Consumers need replay tolerance and monitoring for lag or failures.



ORIGINAL AWS SCENARIO / COMMAND

```
expiresAt: epoch seconds in configured TTL attribute
Read path: exclude logically expired data immediately
Stream event: update projection or publish workflow
Consumer: deduplicate and handle failed batches
```

WHAT HAPPENS · Application validity checks can take effect before physical deletion.

ORIGINAL INTERVIEWER FOLLOW-UP

What recovery plan do you need if a stream consumer falls behind retention?

Answer: Track checkpoint age and alarm before retention expiry. If events are lost, rebuild state using source-of-truth records, exports/backups, or a re-scan and reconcile before resuming.

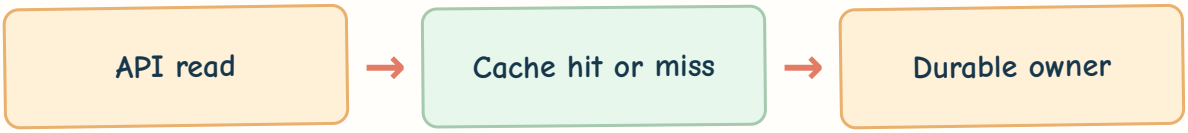
Common mistake: TTL is unsuitable as an exact-time security revocation mechanism by itself.

Source guide p. 35 · Further reading: A28-A30, A59, A67

CHAPTER 11 / 20

Caching and ElastiCache

Accelerate reads without trusting stale authority.



YOUR FIVE INTERVIEW QUESTIONS

051. When would you add ElastiCache to a Java service?

052. What is cache-aside and how do you invalidate entries?

053. What is a cache stampede?

054. What happens when the cache fails?

055. Can Redis-based locks guarantee exactly-once work?

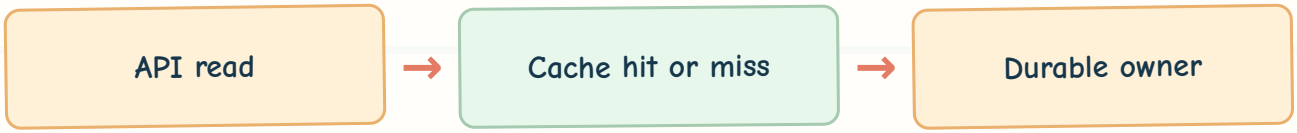
> Say the guarantee and its limit before describing any advanced service.

Q051. When would you add ElastiCache to a Java service?

SAY THIS FIRST · SIMPLE ENGLISH

ElastiCache stores useful temporary data in memory to reduce latency or database reads; the database usually remains the source of truth.

Precise interview detail: Add a compatible managed cache when repeated reads or expensive calculations dominate latency or load and the application can define acceptable freshness. ElastiCache offers engine and deployment choices; verify the selected engine's API, persistence and availability features.



ORIGINAL AWS SCENARIO / COMMAND

```
Request -> look up product key
Cache hit -> return permitted cached representation
Cache miss -> load durable source and populate
Source remains authority for product data
```

WHAT HAPPENS · A cache can reduce database work but introduces stale data and new failure paths.

ORIGINAL INTERVIEWER FOLLOW-UP

Which data in a payment application should you avoid treating as cache-authoritative?

Answer: Do not make cached balances, confirmed payment states, or scarce inventory authoritative. Read and update those with a durable transactional owner; cache only safe views.

Common mistake: Adding a cache before measuring query and index performance may complicate a solvable database issue.

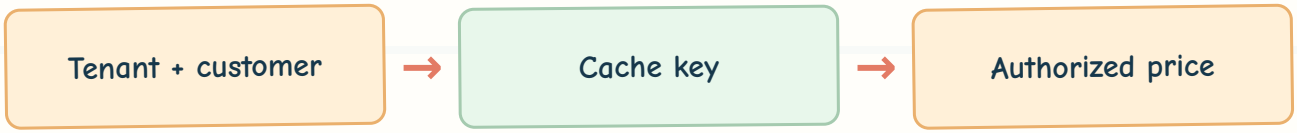
Source guide p. 36 · Further reading: A51, A41

Q052. What is cache-aside and how do you invalidate entries?

SAY THIS FIRST · SIMPLE ENGLISH

Cache-aside first checks a cache and loads from the database on a miss; writes need a safe invalidation plan.

Precise interview detail: In cache-aside, the application loads missing data from the source and populates the cache. After updates, invalidate or refresh affected entries. Define a freshness budget and handle races between readers and writers; TTL bounds some staleness but is not instant coherence.



ORIGINAL AWS SCENARIO / COMMAND

```
Read miss -> DB read -> cache value with TTL
Write -> DB commit -> invalidate affected key
Race -> old reader may repopulate an old value
Mitigation -> versioning or stronger coordination
```

WHAT HAPPENS · Invalidation must consider all derived keys, including lists and aggregates.

ORIGINAL INTERVIEWER FOLLOW-UP

How would you cache customer-specific prices safely?

Answer: Key the cache by tenant and customer context, authorize before returning a price, and include pricing rules or version in the key. Never reuse a shared key for private quotes.

Common mistake: Deleting a key after a commit is not automatically a proof of zero stale reads.

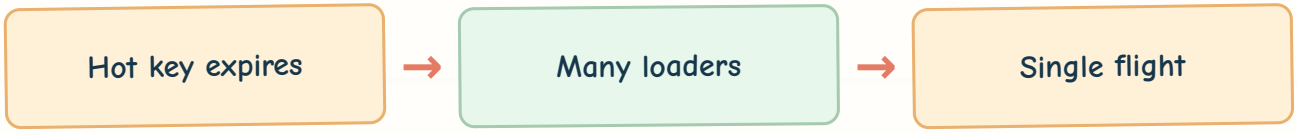
Source guide p. 37 · Further reading: A51, A41

Q053. What is a cache stampede?

SAY THIS FIRST · SIMPLE ENGLISH

A cache stampede happens when many users miss the same hot key together and overload the source database.

Precise interview detail: A stampede occurs when many requests simultaneously rebuild a missing or expired value and overwhelm the source. Use request coalescing, TTL jitter, controlled refresh or bounded stale serving according to the business contract.



ORIGINAL AWS SCENARIO / COMMAND

```
1,000 callers -> same expired product key
Without coordination -> 1,000 DB loads
With coalescing -> one loader, bounded waiters
Add jitter -> spread unrelated expirations
```

WHAT HAPPENS · Protect both loader concurrency and waiting request resources.

ORIGINAL INTERVIEWER FOLLOW-UP

When would stale-while-revalidate be unacceptable?

Answer: Do not serve stale values for a one-time password, permission revocation, payment authorization, or final inventory decision. Those need current authoritative checks.

Common mistake: A distributed lock without a timeout or correct ownership release can turn a stampede into an outage.

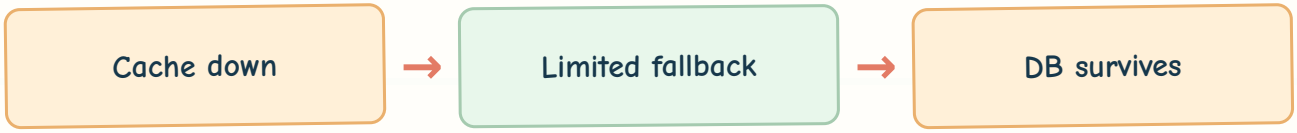
Source guide p. 37 · Further reading: A51, A41

Q054. What happens when the cache fails?

SAY THIS FIRST · SIMPLE ENGLISH

A cache outage should not create unlimited new database requests; use bounded fallback and protect essential operations.

Precise interview detail: Choose a deliberate degraded mode: fall back within a bounded database budget, serve allowed stale data, shed load or fail selected operations. Circuit breakers and timeouts prevent the cache failure from holding all application threads.



ORIGINAL AWS SCENARIO / COMMAND

```
Cache timeout budget: small part of request deadline
Fallback: concurrency-limited DB query
Over budget: reject or degrade selected endpoints
Recovery: avoid synchronized cache refill
```

WHAT HAPPENS · A healthy cache can hide how little database capacity remains for an all-miss event.

ORIGINAL INTERVIEWER FOLLOW-UP

What failure test would verify your fallback budget?

Answer: Disable or simulate Redis failure under representative peak load. Verify database CPU, connection pools, fallback rate limits, p99 latency and error behavior stay within stated budgets.

Common mistake: Unlimited fallback can transform a cache outage into a database outage.

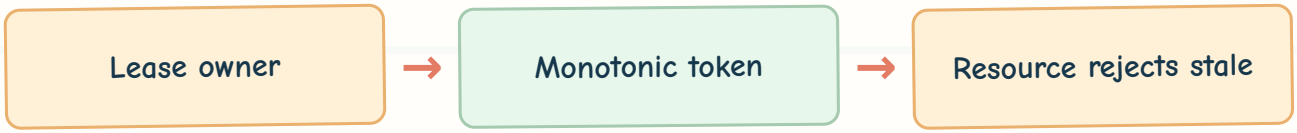
Source guide p. 38 · Further reading: A51, A41

Q055. Can Redis-based locks guarantee exactly-once work?

SAY THIS FIRST · SIMPLE ENGLISH

Redis locks coordinate work but do not alone prove an external effect happens exactly once.

Precise interview detail: A lease-style lock can coordinate attempts under stated assumptions, but expiration, pauses and network failures can allow an old holder to continue. Use durable invariants, idempotency and fencing where necessary. The underlying resource must enforce the correctness boundary.



ORIGINAL AWS SCENARIO / COMMAND

```
Worker A holds lease, then pauses
Lease expires; worker B starts
Worker A resumes with stale authority
Fencing or durable invariant rejects stale effect
```

WHAT HAPPENS · Mutual exclusion during normal conditions is weaker than a durable business guarantee.

ORIGINAL INTERVIEWER FOLLOW-UP

Where would you validate a fencing token?

Answer: The protected database or resource must store the highest accepted fencing token and reject stale tokens in the write path. An unchecked token offers no safety.

Common mistake: A lock is not a substitute for a database uniqueness constraint or payment idempotency key.

Source guide p. 38 · Further reading: A51, A41

CHAPTER 12 / 20

SQS, SNS and EventBridge

Accept replay and recoverable delivery.



YOUR FIVE INTERVIEW QUESTIONS

056. How do SQS, SNS and EventBridge differ?

057. What guarantees do SQS Standard and FIFO queues provide?

058. What is the SQS visibility timeout?

059. What are DLQs and redrive policies?

060. How do you publish an event reliably after a database commit?

> Say the guarantee and its limit before describing any advanced service.

Q056. How do SQS, SNS and EventBridge differ?

SAY THIS FIRST · SIMPLE ENGLISH

SQS holds tasks for workers, SNS fans out published messages, and EventBridge routes matching events by rules.

Precise interview detail: SQS buffers messages for consumers that poll and acknowledge them. SNS publishes to subscribers for fan-out. EventBridge routes events using rules and supported integrations. They can be combined; choose based on buffering, routing, subscriber ownership and failure handling.



ORIGINAL AWS SCENARIO / COMMAND

```
Order service -> event routing or notification
Subscriber A queue -> email worker
Subscriber B queue -> analytics worker
Independent queues -> independent retry and lag
```

WHAT HAPPENS · A queue decouples consumer availability from producer timing.

ORIGINAL INTERVIEWER FOLLOW-UP

Why might every subscriber need its own queue?

Answer: Give each independent subscriber its own queue so one slow consumer cannot block another and each group gets independent retries, retention, dead-letter handling, and scaling.

Common mistake: A topic by itself does not define the same consumer-controlled acknowledgement workflow as an SQS queue.

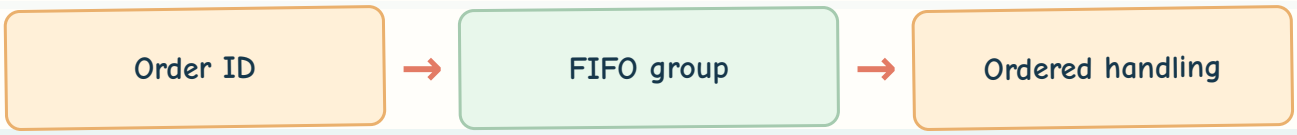
Source guide p. 39 · Further reading: A31-A35, A52

Q057. What guarantees do SQS Standard and FIFO queues provide?

SAY THIS FIRST · SIMPLE ENGLISH

SQS Standard can deliver duplicates and has best-effort ordering. FIFO offers scoped ordering and deduplication rules, not universally once-only business effects.

Precise interview detail: Standard queues support at-least-once delivery and best-effort ordering. FIFO queues provide ordering within message groups and producer deduplication under the documented rules. Neither makes an external business side effect exactly once without application coordination.



ORIGINAL AWS SCENARIO / COMMAND

```
Standard: tolerate duplicate and out-of-order events
FIFO group: order-42 -> sequence for that group
Deduplication ID: stable ID for a producer retry
Consumer: durable business-key deduplication
```

WHAT HAPPENS · A FIFO message can become visible again after an unacknowledged processing attempt.

ORIGINAL INTERVIEWER FOLLOW-UP

What grouping key gives useful parallelism without breaking required ordering?

Answer: Group by an entity requiring order, such as orderId, so independent orders can run in parallel. A single global group would create a processing bottleneck.

Common mistake: Service-level deduplication does not atomically combine 'charge card' and 'delete message'.

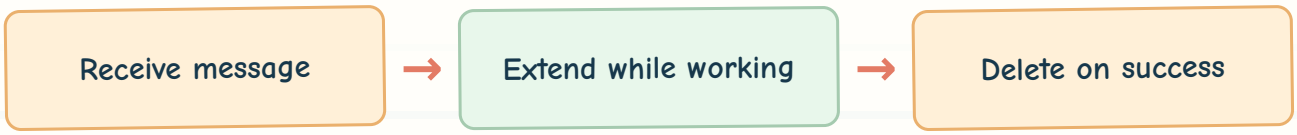
Source guide p. 40 · Further reading: A31-A35, A52

Q058. What is the SQS visibility timeout?

SAY THIS FIRST · SIMPLE ENGLISH

Visibility timeout temporarily hides a received SQS message while a worker processes it; failure to delete can lead to redelivery.

Precise interview detail: After receipt, visibility timeout temporarily hides a message from other consumers. It does not delete the message. Successful processing must be followed by deletion using the receipt handle. Extend visibility for longer work within service constraints and tolerate redelivery.



ORIGINAL AWS SCENARIO / COMMAND

```
Receive -> receipt handle H1 -> hidden interval
Process and commit durable effect
DeleteMessage(H1) after successful commit
Crash before delete -> message can return
```

WHAT HAPPENS · Size the timeout for realistic processing and retries rather than only average duration.

ORIGINAL INTERVIEWER FOLLOW-UP

What is a safe heartbeat strategy for variable-length jobs?

Answer: Extend visibility periodically only while the worker holds valid ownership, with a maximum run time, progress checks, and a recovery policy. Set timeout above normal work with margin.

Common mistake: A short visibility timeout can create concurrent processing; a very long one can delay recovery after a crash.

Source guide p. 40 · Further reading: A31-A35, A52

Q059. What are DLQs and redrive policies?

SAY THIS FIRST · SIMPLE ENGLISH

A dead-letter queue keeps messages that repeatedly fail, giving operators a way to investigate and replay.

Precise interview detail: A dead-letter queue captures eligible messages that exceeded configured processing attempts. It supports diagnosis and controlled recovery. Decide retention, alerting and redrive procedures so failed work is neither silently lost nor endlessly replayed.



ORIGINAL AWS SCENARIO / COMMAND

```
Main queue -> repeated processing failure
Redrive threshold -> dead-letter queue
Operator -> classify cause and repair
Controlled replay -> verify duplicate tolerance
```

WHAT HAPPENS · A DLQ is an operational backlog with ownership and an SLA. For FIFO workflows, removing a failed message can affect the intended business sequence.

ORIGINAL INTERVIEWER FOLLOW-UP

How would you distinguish transient failure from invalid business input?

Answer: Retry network timeouts with bounded backoff; reject schema-invalid or impossible business messages without endless retries. Record event ID, reason, attempts, and repair owner.

Common mistake: Automatically replaying every poison message can recreate the incident.

Source guide p. 41 · Further reading: A31-A35, A52

Q060. How do you publish an event reliably after a database commit?

SAY THIS FIRST · SIMPLE ENGLISH

Write the business update and an outbox event in the same database transaction, then publish events asynchronously.

Precise interview detail: Use a transactional outbox: write the business row and an outbox record in one local transaction, then publish asynchronously. The publisher may repeat delivery, so consumers use idempotent processing. Monitor outbox age and replay safely.



ORIGINAL AWS SCENARIO / COMMAND

```
BEGIN
  insert order row
  insert outbox(order_id, event_id, payload)
COMMIT
Publisher -> send -> mark published with recovery
```

WHAT HAPPENS · The outbox closes the gap between a committed business change and a missing publish attempt.

ORIGINAL INTERVIEWER FOLLOW-UP

What happens if sending succeeds but marking the outbox row fails?

Answer: The event may be sent again after restart because the row still looks pending. Consumers must deduplicate the business effect using a durable event ID.

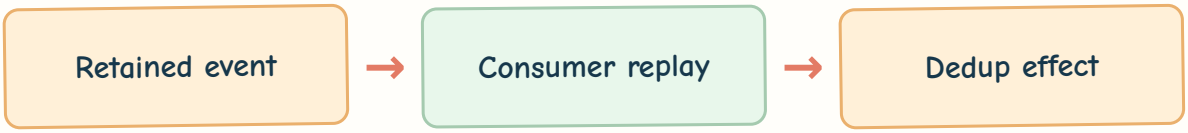
Common mistake: It does not create one distributed transaction spanning the database and queue.

Source guide p. 41 · Further reading: A31-A35, A52

CHAPTER 13 / 20

Streaming and workflow orchestration

Make ordered work and compensations explicit.



YOUR FIVE INTERVIEW QUESTIONS

061. When would you choose Kinesis or MSK instead of SQS?

062. What is partition-key design in streaming?

063. What does Step Functions add to a workflow?

064. What is a saga and how does it differ from a transaction?

065. How do you evolve event schemas safely?

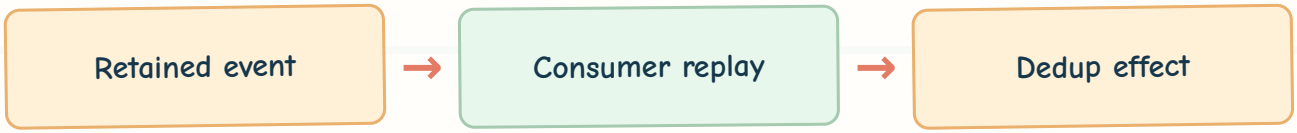
> Say the guarantee and its limit before describing any advanced service.

Q061. When would you choose Kinesis or MSK instead of SQS?

SAY THIS FIRST · SIMPLE ENGLISH

SQS is great for task distribution; Kinesis or MSK fits streaming workloads needing retained ordered records and independent readers.

Precise interview detail: Choose a stream or Kafka-based log when partitioned ordering, replay and multiple independent readers are core needs. SQS fits work distribution with message acknowledgements. Kinesis and MSK have different operational and capacity models; evaluate ecosystem and access patterns.



ORIGINAL AWS SCENARIO / COMMAND

```
Work item completed -> queue acknowledgement
Event history -> stream offsets/checkpoints
Kafka ecosystem requirement -> evaluate MSK
AWS-native stream pipeline -> evaluate Kinesis
```

WHAT HAPPENS · The unit of ordering and consumer progress differs between a queue and a partitioned log.

ORIGINAL INTERVIEWER FOLLOW-UP

How would you replay historical events without repeating external side effects?

Answer: Replay with an event ID and an idempotent consumer effect stored transactionally. For external payments/emails use provider keys, result tracking, or controlled replay filters.

Common mistake: Managed Kafka does not remove partition planning, client configuration or data-retention decisions.

Source guide p. 42 · Further reading: A36, A57, A58

Q062. What is partition-key design in streaming?

SAY THIS FIRST · SIMPLE ENGLISH

Partition keys decide both event ordering scope and how streaming load spreads across partitions.

Precise interview detail: A partition key groups related events onto an ordering boundary and distributes traffic. Balance required order against hot partitions and consumer parallelism. Within-partition order does not establish a global order across all producers and partitions.



ORIGINAL AWS SCENARIO / COMMAND

```
Key = orderId -> per-order ordering
Key = constant -> severe concentration risk
Key = random -> distributes but loses entity grouping
Consumers -> checkpoint after safe processing
```

WHAT HAPPENS · Choose the smallest business entity that needs sequential processing.

ORIGINAL INTERVIEWER FOLLOW-UP

How would you preserve customer ordering while scaling a very large customer?

Answer: Keep per-customer sequence numbers at the owner, partition a large customer's independent substreams, and coordinate or reorder when those streams must later become one ordered view.

Common mistake: Increasing consumer count cannot always overcome a single hot partition's serial workload.

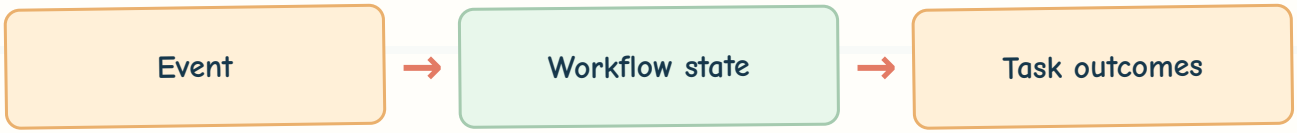
Source guide p. 43 · Further reading: A36, A57, A58

Q063. What does Step Functions add to a workflow?

SAY THIS FIRST · SIMPLE ENGLISH

Step Functions tracks workflow state and transitions across multiple service calls and retries.

Precise interview detail: Step Functions models workflow state, branching, retries and service integrations. Workflow types have different execution and delivery semantics. Keep activities idempotent, use bounded retries and represent compensation explicitly for multi-service business operations.



ORIGINAL AWS SCENARIO / COMMAND

```
Reserve inventory -> authorize payment -> confirm order
On payment rejection -> release inventory
On timeout -> inspect outcome before compensation
Workflow state -> correlate business operation ID
```

WHAT HAPPENS · An explicit workflow improves visibility into progress and recovery.

ORIGINAL INTERVIEWER FOLLOW-UP

When would you choose a Standard workflow over an Express workflow?

Answer: Use Standard for durable long-running workflows with their required audit and execution semantics; use Express for suitable high-volume shorter workflows after checking limits and delivery behavior.

Common mistake: State-machine execution semantics do not make each external API effect magically transactional.

Source guide p. 43 · Further reading: A36, A57, A58

Q064. What is a saga and how does it differ from a transaction?

SAY THIS FIRST · SIMPLE ENGLISH

A saga coordinates several local transactions with recovery or compensation. A single ACID transaction cannot cover arbitrary remote services.

Precise interview detail: A saga coordinates local transactions and compensating actions across services. It exposes intermediate states and compensation failures. A local ACID transaction protects one database boundary; a saga handles a wider business process with weaker atomicity.



ORIGINAL AWS SCENARIO / COMMAND

```
Inventory reservation -> local commit
Payment authorization -> local commit
Failure -> release or refund where appropriate
Customer view -> show pending, completed or failed
```

WHAT HAPPENS · Compensation is a business action, not always a perfect reversal.

ORIGINAL INTERVIEWER FOLLOW-UP

How would you reconcile a payment that succeeded after the workflow timed out?

Answer: Query the provider by stable payment key, match it to order/reservation state, then settle, refund, or compensate according to a documented state machine. Never assume timeout means failure.

Common mistake: Sending an email cannot generally be undone by rolling back another database.

Source guide p. 44 · Further reading: A36, A57, A58

Q065. How do you evolve event schemas safely?

SAY THIS FIRST · SIMPLE ENGLISH

Keep event changes backward-compatible during a rolling migration, and test old consumers against new producers.

Precise interview detail: Use explicit versioning, compatible additions and consumer-tolerant parsing. Validate contracts in tests and keep replay requirements in mind. Deprecate fields only after proving that live and recovery consumers no longer depend on them.



ORIGINAL AWS SCENARIO / COMMAND

```
Event: type=OrderCreated, schemaVersion=2
Add: optional deliveryPreference
Do not reuse: existing field with changed meaning
Replay: old events remain interpretable
```

WHAT HAPPENS · Event retention makes compatibility a historical concern, not just a rollout concern.

ORIGINAL INTERVIEWER FOLLOW-UP

How would you migrate an enum value that older consumers cannot parse?

Answer: Add a compatible fallback or new field/version, teach consumers to handle the new value before producers emit it, and retire legacy behavior after measured adoption.

Common mistake: Updating every live consumer does not guarantee an old event can still be replayed correctly.

Source guide p. 44 · Further reading: A36, A57, A58

CHAPTER 14 / 20

Observability and incident diagnosis

Start with customer-visible evidence.



YOUR FIVE INTERVIEW QUESTIONS

066. How do CloudWatch, CloudTrail and AWS Config differ?

067. Which metrics matter for a Spring Boot API?

068. How should you design alerts and SLOs?

069. How do correlation IDs and distributed tracing help?

070. How would you investigate a sudden production latency increase?

> Say the guarantee and its limit before describing any advanced service.

Q066. How do CloudWatch, CloudTrail and AWS Config differ?

SAY THIS FIRST · SIMPLE ENGLISH

CloudWatch tracks operations and metrics, CloudTrail records supported AWS API activity, and AWS Config tracks resource configuration changes.

Precise interview detail: CloudWatch supports operational telemetry such as metrics, logs and alarms. CloudTrail records supported account activity and API events. AWS Config records supported resource configuration history and evaluates rules. These answer different questions and require appropriate configuration.



ORIGINAL AWS SCENARIO / COMMAND

```
CloudWatch: did API latency increase?
CloudTrail: who changed the security group?
Config: how did the resource configuration evolve?
Trace: where did one request spend its time?
```

WHAT HAPPENS · Combine signals to connect a change with its operational consequences.

ORIGINAL INTERVIEWER FOLLOW-UP

Which signal would investigate unexpected S3 object reads?

Answer: Check CloudTrail S3 data events if enabled for object access, plus relevant CloudWatch logs and access telemetry. Management events alone may not show every GetObject.

Common mistake: CloudTrail data events and other detailed coverage are not all enabled simply because a default event history exists.

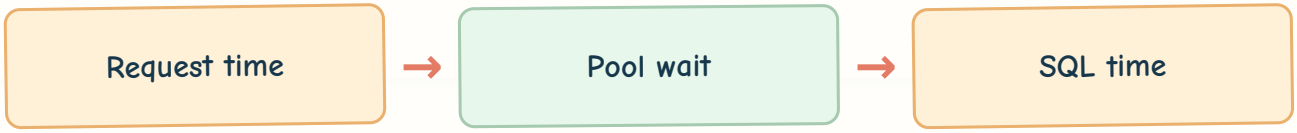
Source guide p. 45 · Further reading: A39-A41

Q067. Which metrics matter for a Spring Boot API?

SAY THIS FIRST · SIMPLE ENGLISH

Track real API success, p95/p99 latency, throughput, saturation, JVM pressure, and downstream delays.

Precise interview detail: Measure customer-facing request rate, errors and latency along with saturation and dependency signals. Track JVM, pool and queue behavior to explain symptoms. Use percentiles and realistic traffic segmentation rather than averages alone.



ORIGINAL AWS SCENARIO / COMMAND

```
API: rate, p95/p99, error ratio
JVM: allocation, GC pause, threads, memory
DB: pool wait, query latency, active sessions
Queue: age, backlog, processing failures
```

WHAT HAPPENS · A rising p99 with normal average latency can indicate a subset of stalled requests.

ORIGINAL INTERVIEWER FOLLOW-UP

Which metric separates database execution time from pool acquisition wait?

Answer: Measure HikariCP connection acquisition wait separately from query execution timer or database traces. A long checkout wait with short SQL means the pool may be exhausted.

Common mistake: A CPU alarm alone may miss thread starvation or a saturated connection pool.

Source guide p. 45 · Further reading: A39-A41

Q068. How should you design alerts and SLOs?

SAY THIS FIRST · SIMPLE ENGLISH

An SLI measures user-visible service behavior; an SLO is its target. Alert on rapid error-budget loss rather than arbitrary noise.

Precise interview detail: Define a service-level objective around customer outcomes and use alerts that demand an actionable response. Monitor error-budget consumption and relevant dependencies. Distinguish immediate paging from lower-urgency capacity or configuration investigation.



ORIGINAL AWS SCENARIO / COMMAND

```
SLO example: 99.9% eligible requests succeed
Window: chosen business measurement period
Alert: excessive error-budget burn
Runbook: diagnosis, mitigation and escalation
```

WHAT HAPPENS · An SLO includes explicit eligibility and measurement definitions.

ORIGINAL INTERVIEWER FOLLOW-UP

Would an expected validation error count against your availability objective?

Answer: An expected 400 validation response normally should not count as unavailable when it is correct behavior. Define eligibility and failures clearly; 500s or overload errors usually do count.

Common mistake: A high dashboard uptime can hide broken critical business operations.

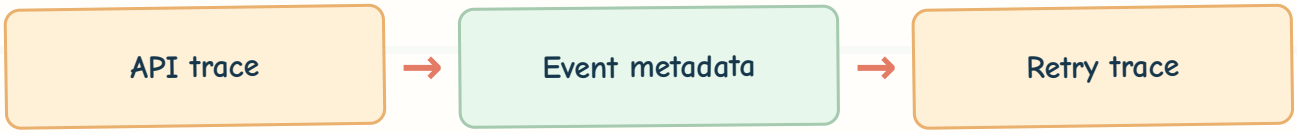
Source guide p. 46 · Further reading: A39-A41

Q069. How do correlation IDs and distributed tracing help?

SAY THIS FIRST · SIMPLE ENGLISH

Correlation IDs connect an API call with downstream logs and traces so one user journey is easier to debug.

Precise interview detail: They connect work across service boundaries and asynchronous stages. Propagate trace context through supported mechanisms and carry business operation identifiers for durable diagnosis. Keep sensitive data out of logs and control metric label cardinality.



ORIGINAL AWS SCENARIO / COMMAND

```
HTTP request -> trace context
Outbox event -> eventId and business operation ID
Worker -> linked trace or propagated context
Logs -> timestamp, service, operation, outcome
```

WHAT HAPPENS · A trace identifies timing relationships; a business ID connects retries and delayed workflow work.

ORIGINAL INTERVIEWER FOLLOW-UP

How would you correlate a queue retry with the original API request?

Answer: Include the original trace/correlation ID in event metadata, preserve message ID and attempt count, and start a linked consumer span on each retry. Do not use the trace ID as an idempotency key.

Common mistake: Customer IDs as unbounded metric labels can create high cardinality and cost.

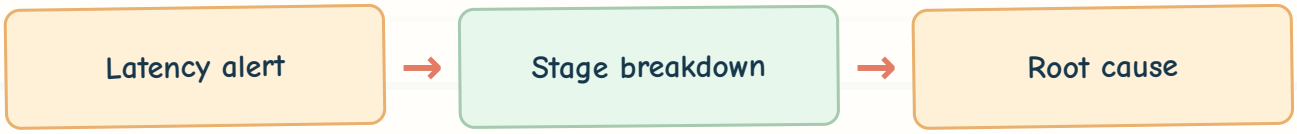
Source guide p. 46 · Further reading: A39-A41

Q070. How would you investigate a sudden production latency increase?

SAY THIS FIRST · SIMPLE ENGLISH

Diagnose latency by finding where time grew: network, load balancer, JVM, database, or external APIs.

Precise interview detail: Establish the impact and time window, compare recent changes, then localize delay using ingress, application and dependency evidence. Apply a bounded mitigation and verify customer recovery before deeper optimization.



ORIGINAL AWS SCENARIO / COMMAND

```
1. Check error ratio and latency by endpoint
2. Compare deploy, config and traffic changes
3. Inspect traces, pools, GC and DB waits
4. Mitigate; validate; preserve evidence
```

WHAT HAPPENS · Evidence distinguishes a capacity issue from a regression or dependency failure.

ORIGINAL INTERVIEWER FOLLOW-UP

What would you do if rollback does not reduce the latency?

Answer: If rollback has no effect, look for shared dependency saturation, regional incidents, queueing, a hot DB query, DNS issues, and network changes. Use traces and time-series evidence before scaling.

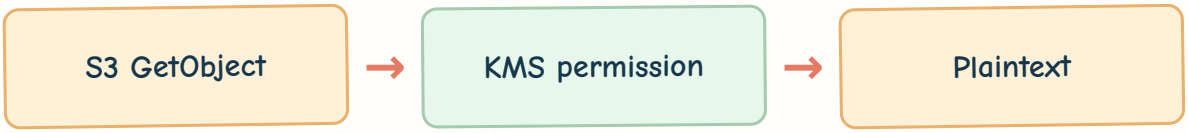
Common mistake: Restarting every task before collecting diagnostics can erase the information needed to prevent recurrence.

Source guide p. 47 · Further reading: A39-A41

CHAPTER 15 / 20

KMS, secrets and application security

Protect credentials and tenant boundaries.



YOUR FIVE INTERVIEW QUESTIONS

071. What is KMS and what is envelope encryption?

072. How do Secrets Manager and Parameter Store differ?

073. Should secrets be injected at startup or retrieved at runtime?

074. How do Cognito and IAM differ for a web application?

075. How do you secure a multi-tenant AWS application?

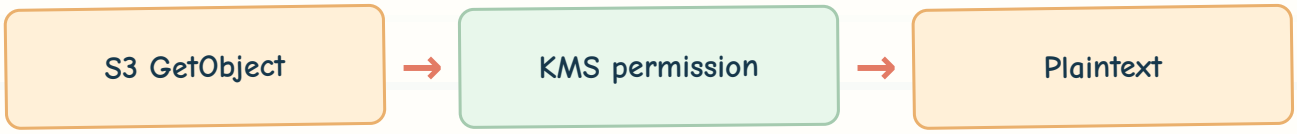
> Say the guarantee and its limit before describing any advanced service.

Q071. What is KMS and what is envelope encryption?

SAY THIS FIRST · SIMPLE ENGLISH

KMS manages cryptographic keys. Envelope encryption encrypts data with a data key and protects that key using a KMS key.

Precise interview detail: KMS manages keys and cryptographic operations under policy controls. Envelope encryption encrypts data with a data key, then protects that key with a higher-level key. Service integrations can manage this workflow, while authorization controls who can use the key.



ORIGINAL AWS SCENARIO / COMMAND

```
Plain data -> encrypt with data key
Data key -> encrypt under KMS key
Store ciphertext + encrypted data key
Read -> authorized key access -> decrypt data
```

WHAT HAPPENS · Protecting encrypted data requires both secure key use and secure data access.

ORIGINAL INTERVIEWER FOLLOW-UP

What happens if an application's S3 access works but KMS decrypt is denied?

Answer: S3 permission may succeed but downloading a KMS-encrypted object can fail because the caller lacks effective kms:Decrypt rights or the key policy/conditions reject it.

Common mistake: Key rotation is not the same as rewriting every old object with a new data key.

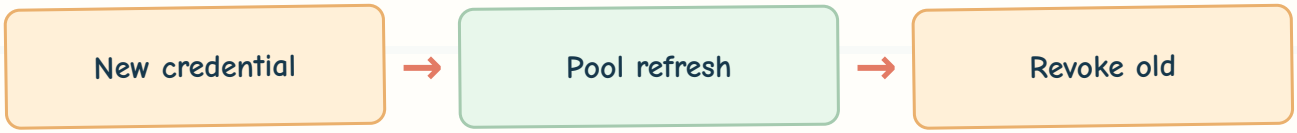
Source guide p. 48 · Further reading: A37, A38, A48, A60, A66

Q072. How do Secrets Manager and Parameter Store differ?

SAY THIS FIRST · SIMPLE ENGLISH

Secrets Manager specializes in managed secrets and rotation; Parameter Store stores configuration and can hold protected values.

Precise interview detail: Secrets Manager provides secret management features including supported rotation workflows. Systems Manager Parameter Store stores hierarchical configuration with supported encrypted SecureString values. Compare rotation, integration, throughput, size and cost requirements rather than treating them as identical products.



ORIGINAL AWS SCENARIO / COMMAND

```
Nonsecret feature setting -> configuration parameter
Database password -> managed secret workflow
Application retrieval -> IAM + KMS where applicable
Cache -> controlled refresh and failure behavior
```

WHAT HAPPENS · Choose a lifecycle that matches how the value changes and how applications consume it.

ORIGINAL INTERVIEWER FOLLOW-UP

How would you keep a connection pool working during rotation?

Answer: Allow old and new credentials to overlap, update the secret, reconnect or refresh existing pool connections, verify success, then revoke old credentials after all replicas transition.

Common mistake: A stored encrypted password does not automatically rotate the database credential.

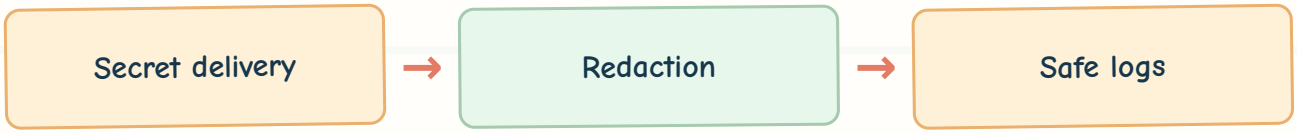
Source guide p. 48 · Further reading: A37, A38, A48, A60, A66

Q073. Should secrets be injected at startup or retrieved at runtime?

SAY THIS FIRST · SIMPLE ENGLISH

Startup secret injection is simple but needs restart for changes. Runtime retrieval permits refresh but adds an API and caching contract.

Precise interview detail: Startup injection is simple but normally requires a new workload deployment to consume a changed value. Runtime retrieval can support refresh but adds dependency, cache and authorization behavior. Select the strategy from rotation timing and failure requirements.



ORIGINAL AWS SCENARIO / COMMAND

```
ECS configured secret -> resolved for task startup
Java SDK retrieval -> application fetches and caches
Rotation -> update consumer at defined time
Fallback -> use only a still-valid allowed value
```

WHAT HAPPENS · The consumer must know when a retrieved secret stops being valid.

ORIGINAL INTERVIEWER FOLLOW-UP

How would you prevent credentials from appearing in startup logs?

Answer: Never print environment dumps, command-line credentials, connection URLs with passwords, or exception values containing secrets. Redact known keys and test logs during startup failures.

Common mistake: Rotating the source does not automatically update an already injected environment value.

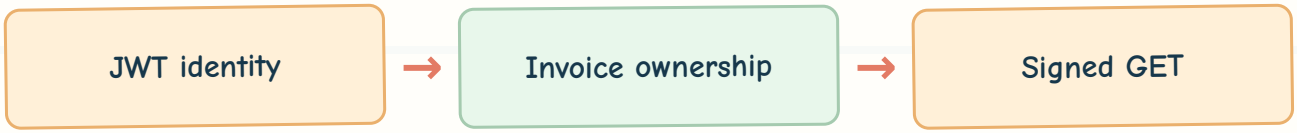
Source guide p. 49 · Further reading: A37, A38, A48, A60, A66

Q074. How do Cognito and IAM differ for a web application?

SAY THIS FIRST · SIMPLE ENGLISH

Cognito can authenticate application users; IAM secures AWS API actions by principals. Neither automatically applies your business ownership rules.

Precise interview detail: Cognito can provide application user authentication and token-related capabilities. IAM governs access to AWS resources. For a Spring resource server, validate the intended token type, issuer, signature, expiry and audience or client-related constraints appropriate to the token contract.



ORIGINAL AWS SCENARIO / COMMAND

```
User login -> identity provider -> access token
Spring API -> validate token -> authorize resource
Java workload -> IAM role -> S3 or SQS
User identity != workload service credentials
```

WHAT HAPPENS · An authenticated user still needs ownership or role checks for each business operation.

ORIGINAL INTERVIEWER FOLLOW-UP

How would you ensure customer A cannot read customer B's invoice?

Answer: Validate the user token and then check invoice owner or tenant ID in your database before issuing a scoped S3 download. A valid token alone does not authorize another person's invoice.

Common mistake: Accepting an ID token as an API access token can confuse authentication purpose and authorization.

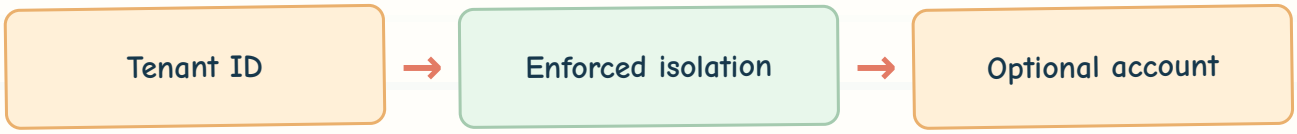
Source guide p. 49 · Further reading: A37, A38, A48, A60, A66

Q075. How do you secure a multi-tenant AWS application?

SAY THIS FIRST · SIMPLE ENGLISH

Secure tenants at the API, identity, data query, storage, and observability layers.

Precise interview detail: Enforce tenant identity at trusted boundaries and in data access. Scope object keys, queries and credentials appropriately, and verify negative tests. Choose shared or isolated infrastructure based on compliance, noisy-neighbor, operational and cost requirements.



ORIGINAL AWS SCENARIO / COMMAND

```
Token tenant -> trusted authorization context
SQL -> predicate includes tenant_id
S3 -> bounded tenant prefix access design
Audit -> tenant, actor, operation, result
```

WHAT HAPPENS · Infrastructure boundaries complement application-level ownership checks.

ORIGINAL INTERVIEWER FOLLOW-UP

What requirement would justify a separate account per tenant?

Answer: Use separate accounts when isolation, regulatory boundaries, independent keys, audit ownership, quota separation, or customer contracts require stronger blast-radius boundaries than pooled tenancy.

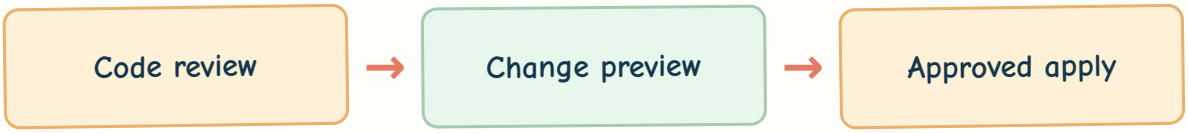
Common mistake: A tenant ID supplied only in a request body must not override the authenticated authorization context.

Source guide p. 50 · Further reading: A37, A38, A48, A60, A66

CHAPTER 16 / 20

Infrastructure as code and CI/CD

Build once, review changes, release safely.



YOUR FIVE INTERVIEW QUESTIONS

076. What is infrastructure as code?

077. How do CloudFormation and CDK differ?

078. How should a Java container pipeline work?

079. How do rolling, blue-green and canary deployments differ?

080. How should CI authenticate to AWS?

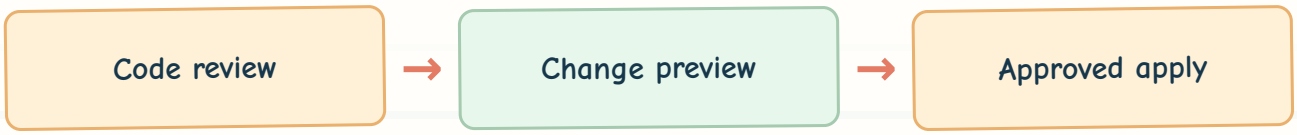
> Say the guarantee and its limit before describing any advanced service.

Q076. What is infrastructure as code?

SAY THIS FIRST · SIMPLE ENGLISH

Infrastructure as code stores reviewed cloud configuration in versioned files instead of unmanaged console clicks.

Precise interview detail: Infrastructure as code describes resources and changes in versioned definitions. CloudFormation, CDK and other tools offer different authoring and state models. Review generated plans, validate security and detect drift; declarative definitions do not remove deployment risk.



ORIGINAL AWS SCENARIO / COMMAND

```
Commit -> review -> validation -> change preview
Apply -> service deployment -> verification
Record -> resource identity and artifact digest
Drift -> compare actual state with intended state
```

WHAT HAPPENS · Repeatability depends on parameters, permissions, external dependencies and tool behavior.

ORIGINAL INTERVIEWER FOLLOW-UP

Which changes would you prevent from being automatically applied?

Answer: Require manual approval for destructive data changes, broad IAM permission increases, production network exposure, key deletion, and other high-impact changes. Preview the proposed change set.

Common mistake: A template that parses is not proof that deployment will succeed or be safe.

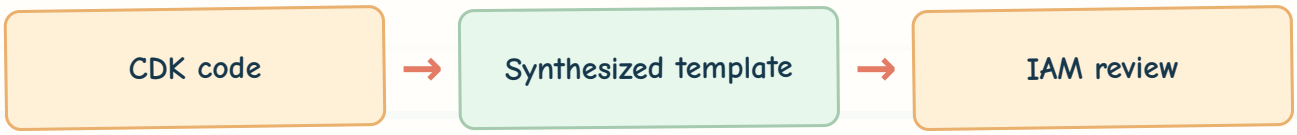
Source guide p. 51 · Further reading: A43, A47, A48

Q077. How do CloudFormation and CDK differ?

SAY THIS FIRST · SIMPLE ENGLISH

CloudFormation runs declarative resource templates; CDK generates those templates from general-purpose code.

Precise interview detail: CloudFormation deploys AWS resource templates through stacks and managed lifecycle operations. CDK lets you author infrastructure using programming languages and synthesizes CloudFormation templates for its common deployment model. Both require reviewing the actual resource changes.



ORIGINAL AWS SCENARIO / COMMAND

```
CDK source -> synthesize template
CloudFormation template -> change set -> stack update
Parameters and context -> deployment-specific inputs
Application image -> separate immutable artifact
```

WHAT HAPPENS · Higher-level constructs can accelerate authoring while producing resources that still need inspection.

ORIGINAL INTERVIEWER FOLLOW-UP

Why should a reviewer inspect synthesized IAM policies?

Answer: Synthesized templates show the actual IAM actions, resources, wildcards, and trust conditions. Reviewers should not trust friendly source-code constructs without inspecting the effective policy.

Common mistake: A short CDK construct can create many resources and permissions.

Source guide p. 52 · Further reading: A43, A47, A48

Q078. How should a Java container pipeline work?

SAY THIS FIRST · SIMPLE ENGLISH

A delivery pipeline should test Java, build once, scan and sign the image as needed, publish its digest, and deploy that exact artifact.

Precise interview detail: Build and test one immutable candidate, scan and verify it, publish its digest, deploy that digest and run health and business smoke checks. Keep environment configuration separate from the binary and capture the identity needed for rollback.



ORIGINAL AWS SCENARIO / COMMAND

```
Source -> unit/integration tests -> image candidate
Candidate -> security and runtime checks -> registry
Digest -> stage -> approval policy -> production
Observe -> compare error, latency and business result
```

WHAT HAPPENS · Promoting the same digest reduces accidental differences between stages.

ORIGINAL INTERVIEWER FOLLOW-UP

What should a production smoke test verify beyond HTTP 200?

Answer: Smoke-test representative authenticated business paths, database connectivity, permission boundaries, real writes with safe fixtures, telemetry, and failure handling—not just a health URL.

Common mistake: Rebuilding separately for production can change dependencies while keeping the same source commit.

Source guide p. 52 · Further reading: A43, A47, A48

Q079. How do rolling, blue-green and canary deployments differ?

SAY THIS FIRST · SIMPLE ENGLISH

Rolling replaces instances gradually, blue-green swaps between separate fleets, and canary exposes a small share to the new version.

Precise interview detail: Rolling replaces portions of existing capacity. Blue-green prepares a separate environment or target set and switches traffic. Canary exposes a small share of traffic to the new version before expansion. Each needs spare capacity, compatibility and a measurable rollback decision.



ORIGINAL AWS SCENARIO / COMMAND

```
Rolling -> mixed old/new versions
Blue-green -> switch target set
Canary -> limited traffic -> evaluate -> expand
Every approach -> compatible schema and events
```

WHAT HAPPENS · Traffic switching alone cannot reverse incompatible database writes.

ORIGINAL INTERVIEWER FOLLOW-UP

Which signals would stop an automated rollout?

Answer: Stop rollout if customer-visible error rate, p99, dependency pressure, or critical business invariant alerts breach guardrails. Consider a minimum sample size and observation period.

Common mistake: A canary without enough representative requests may miss a critical regression.

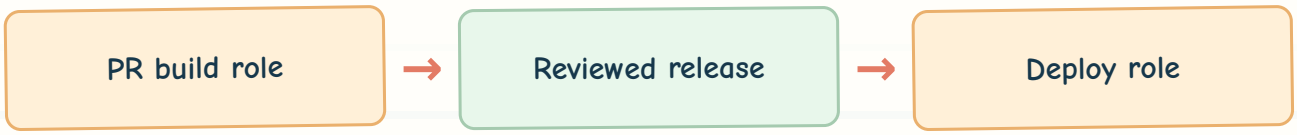
Source guide p. 53 · Further reading: A43, A47, A48

Q080. How should CI authenticate to AWS?

SAY THIS FIRST · SIMPLE ENGLISH

CI should obtain temporary workload credentials using an approved federation or role assumption flow, not store permanent AWS keys.

Precise interview detail: Use a supported identity federation mechanism to obtain temporary deployment-role credentials. Restrict role trust to the expected repository, workflow, environment or equivalent claims, and bound deployment permissions. Keep build and production roles separate where practical.



ORIGINAL AWS SCENARIO / COMMAND

```
CI identity token -> STS federation -> deploy role
Trust conditions -> intended workflow identity
Role policy -> required deployment operations
Session -> temporary credentials and audit trail
```

WHAT HAPPENS · Trust claims protect who can obtain the role, while permissions restrict what it can do.

ORIGINAL INTERVIEWER FOLLOW-UP

How would you separate pull-request builds from production deployment authority?

Answer: Do not give untrusted pull requests production-role trust. Separate build identity from release identity and require reviewed branch, protected environment and tightly scoped deploy role.

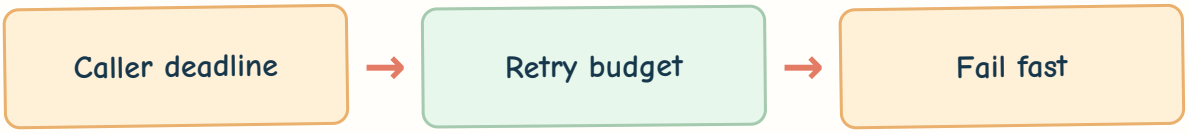
Common mistake: A wildcard trust condition can let an unintended workflow deploy even with no stored access key.

Source guide p. 53 · Further reading: A43, A47, A48

CHAPTER 17 / 20

Resilience, retries and disaster recovery

Define what remains correct under partial failure.



YOUR FIVE INTERVIEW QUESTIONS

081. How do timeouts, retries and circuit breakers work together?

082. What is idempotency in an AWS application?

083. What are common AWS disaster recovery strategies?

084. Why is multi-Region active-active difficult?

085. What failure experiments would you run before production?

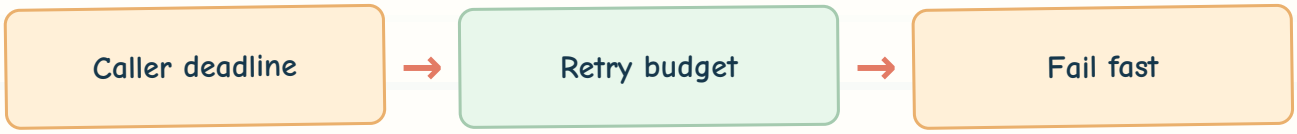
> Say the guarantee and its limit before describing any advanced service.

Q081. How do timeouts, retries and circuit breakers work together?

SAY THIS FIRST · SIMPLE ENGLISH

Timeouts bound waiting; retries address likely temporary failures; circuit breakers reject calls when a dependency is unhealthy.

Precise interview detail: A timeout bounds waiting. A retry starts another attempt where semantics allow it. A circuit breaker temporarily avoids a failing dependency based on a policy. Combine an end-to-end deadline, limited attempts, jitter and concurrency bounds.



ORIGINAL AWS SCENARIO / COMMAND

```
Total request deadline: 1,000 ms
Attempts: at most 2 with bounded backoff
Per-attempt timeout: fit remaining budget
Retry eligibility: transient + idempotent or resolvable
```

WHAT HAPPENS · Multiple layers of retry can multiply attempts and overload a recovering service.

ORIGINAL INTERVIEWER FOLLOW-UP

How would you prevent SDK retries and service retries from exceeding the caller's deadline?

Answer: Set one end-to-end deadline and pass remaining time to each layer. Limit total retries, use jitter, and align SDK settings with the caller's budget to avoid retry storms.

Common mistake: A timeout does not prove that the downstream operation did not complete.

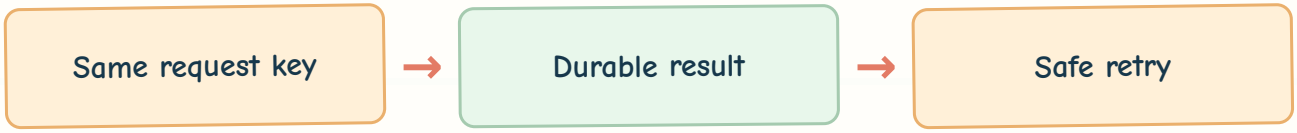
Source guide p. 54 · Further reading: A35, A41, A42

Q082. What is idempotency in an AWS application?

SAY THIS FIRST · SIMPLE ENGLISH

Idempotency means repeating a request has one intended business effect when the caller reuses the same durable key.

Precise interview detail: Idempotency means repeating a logical operation has the same intended effect as one successful execution. Implement it with a stable operation key, durable state and recovery for uncertain outcomes. Include result replay and key-retention policy.



ORIGINAL AWS SCENARIO / COMMAND

```
Client key -> unique durable operation row
First request -> execute and record outcome
Repeat -> return outcome or in-progress status
Different payload with same key -> reject conflict
```

WHAT HAPPENS · The request key must cover the business operation rather than one network attempt.

ORIGINAL INTERVIEWER FOLLOW-UP

How long should payment idempotency records remain available?

Answer: Retain payment keys longer than your maximum realistic client and provider retry/reconciliation window, considering delayed messages and business records. Document scope and expiry explicitly.

Common mistake: A randomly generated key on every retry removes the deduplication benefit.

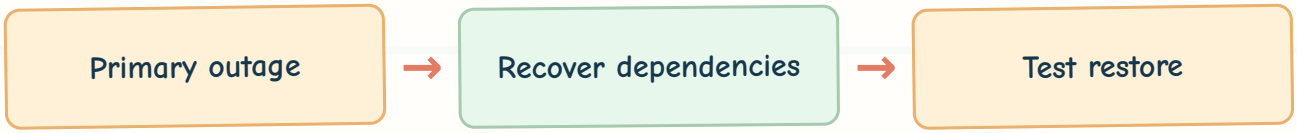
Source guide p. 55 · Further reading: A35, A41, A42

Q083. What are common AWS disaster recovery strategies?

SAY THIS FIRST · SIMPLE ENGLISH

Disaster recovery choices range from backup-and-restore to warm standby and active-active, with different RTO, RPO, and cost.

Precise interview detail: Common patterns include backup-and-restore, pilot light, warm standby and multi-site designs. They trade operational cost and complexity against achievable recovery objectives. Verify data replication, infrastructure readiness, dependencies and traffic cutover through rehearsals.



ORIGINAL AWS SCENARIO / COMMAND

```
Backup/restore -> rebuild and recover copies
Pilot light -> essential foundations retained
Warm standby -> reduced running environment
Multi-site -> coordinated serving and data design
```

WHAT HAPPENS · Architecture labels do not guarantee a recovery time; measured drills establish evidence.

ORIGINAL INTERVIEWER FOLLOW-UP

Which dependencies belong in your DR checklist?

Answer: Include DNS, IAM/KMS keys, secrets, certificates, ECR images, app config, VPC, database, S3 data, event queues, client integrations, and runbooks in the DR inventory.

Common mistake: An active application in another Region is not useful if its critical secrets or data cannot be recovered.

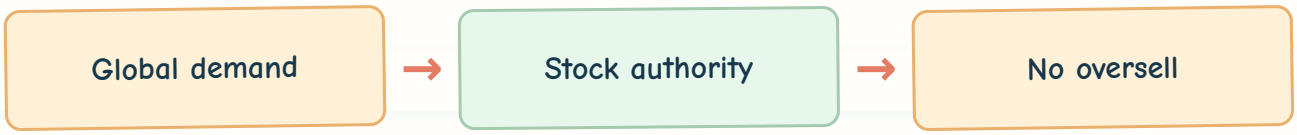
Source guide p. 55 · Further reading: A35, A41, A42

Q084. Why is multi-Region active-active difficult?

SAY THIS FIRST · SIMPLE ENGLISH

Active-active across Regions is hard because state, ordering, failover, identity, and conflicts become distributed problems.

Precise interview detail: Serving from multiple Regions adds data-conflict handling, routing, identity, capacity and operational coordination. Decide which operations can be independent and which require a single authority or a suitable consistency mechanism. Higher availability can come with harder correctness tradeoffs.



ORIGINAL AWS SCENARIO / COMMAND

```
Catalog reads -> distributed copies may be sufficient
Inventory decrement -> bounded global invariant
Payment -> external authority and idempotent outcome
Failover -> prevent unintended competing writers
```

WHAT HAPPENS · Business invariants determine whether stale or conflicting writes are acceptable.

ORIGINAL INTERVIEWER FOLLOW-UP

How would you prevent overselling a scarce product across Regions?

Answer: Assign scarce stock to one authority or divide stock into region-specific allocations whose sum never exceeds real stock. A stale global cache must not decide the last item.

Common mistake: 'Use global tables' is not a complete answer to global stock correctness.

Source guide p. 56 · Further reading: A35, A41, A42

Q085. What failure experiments would you run before production?

SAY THIS FIRST · SIMPLE ENGLISH

Failure tests prove whether timeouts, queues, recovery, and rate limits behave as designed rather than only on paper.

Precise interview detail: Test the stated failure model with bounded experiments and recovery assertions. Include task loss, dependency slowdown, cache loss, database failover, duplicate events and restore. Measure customer impact, backlog recovery and operational response.



ORIGINAL AWS SCENARIO / COMMAND

```
Kill one task -> target recovery without lost invariant
Duplicate message -> one business effect
DB failover -> bounded errors and eventual recovery
Restore -> validated data and working application
```

WHAT HAPPENS · A good experiment includes a stop condition, observations and cleanup of only its own test resources.

ORIGINAL INTERVIEWER FOLLOW-UP

Which experiment would expose an unlimited fallback from cache to database?

Answer: Cut the cache during peak replay traffic and watch DB connections, admission limits, rejection rate, pool wait and latency. The test fails if fallback traffic overwhelms the database.

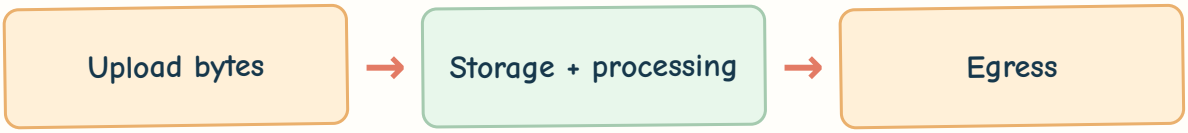
Common mistake: Testing only happy-path load provides no evidence of recovery behavior.

Source guide p. 56 · Further reading: A35, A41, A42

CHAPTER 18 / 20

Cost, quotas and performance

Measure usage, quotas, and tradeoffs.



YOUR FIVE INTERVIEW QUESTIONS

086. How should you estimate AWS cost?

087. What is the difference between vertical and horizontal scaling?

088. What are service quotas and why do they matter?

089. How do you reduce observability and storage cost safely?

090. When is serverless more expensive or less suitable?

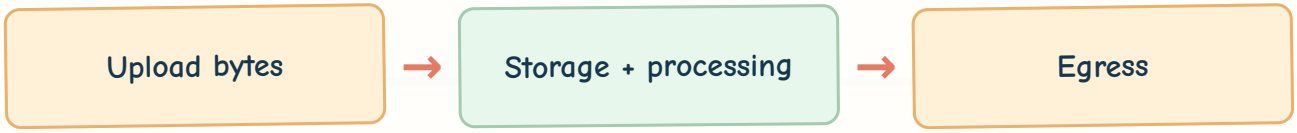
> Say the guarantee and its limit before describing any advanced service.

Q086. How should you estimate AWS cost?

SAY THIS FIRST · SIMPLE ENGLISH

Estimate AWS cost from realistic usage, including data transfer, requests, runtime, retention, and management overhead.

Precise interview detail: Model measured usage and architecture-specific cost components. Include compute, storage, requests, data transfer, logs, backup, networking and managed-service charges. Use current regional pricing and forecast ranges rather than memorized totals.



ORIGINAL AWS SCENARIO / COMMAND

```
Monthly model = baseline + variable usage + resilience
Example drivers: task hours, DB hours, S3 bytes
Often missed: NAT processing, cross-AZ traffic, logs
Compare: cost per successful business operation
```

WHAT HAPPENS · A low compute bill can hide expensive transfer and telemetry paths.

ORIGINAL INTERVIEWER FOLLOW-UP

Which component could become the dominant cost for large uploads?

Answer: Large uploads can be dominated by data transfer, S3 storage and requests, replication, image processing, or outbound CDN traffic depending on direction and scale. Measure bytes by path.

Common mistake: Free-tier assumptions and old prices should not drive production budgets.

Source guide p. 57 · Further reading: A41, A51, A61

Q087. What is the difference between vertical and horizontal scaling?

SAY THIS FIRST · SIMPLE ENGLISH

Vertical scaling makes one machine bigger. Horizontal scaling runs more machines while shared resources may remain fixed.

Precise interview detail: Vertical scaling increases resources of a unit; horizontal scaling adds units. Horizontal compute scaling needs shared or externalized state and bounded downstream capacity. Neither strategy fixes an invariant that serializes every request.



ORIGINAL AWS SCENARIO / COMMAND

```
Vertical: larger database or task
Horizontal: more API tasks behind load balancer
Constraint: DB connections and write contention
Measure: throughput gain versus extra cost
```

WHAT HAPPENS · Stateless application replicas are easier to add and replace than stateful singleton workloads.

ORIGINAL INTERVIEWER FOLLOW-UP

How would you recognize that the database is now the limiting tier?

Answer: Watch database CPU, active sessions, connection waits, lock time, query latency, and flat throughput as API replicas grow. More servers may amplify a database bottleneck.

Common mistake: Doubling replicas does not guarantee doubled end-to-end capacity.

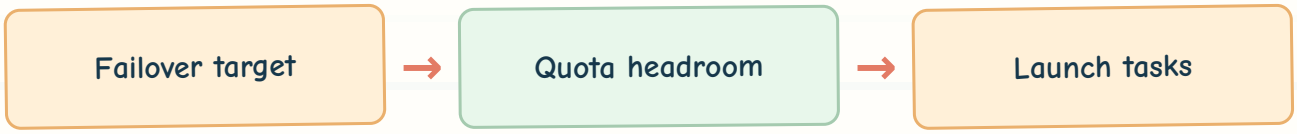
Source guide p. 57 · Further reading: A41, A51, A61

Q088. What are service quotas and why do they matter?

SAY THIS FIRST · SIMPLE ENGLISH

AWS services have regional and account quotas that can block a scale-out or recovery plan.

Precise interview detail: Quotas limit supported resources or request rates and vary by service, account and Region. Some can be adjusted, others are fixed. Include quota review and capacity validation in planning, especially for traffic spikes and recovery into another Region.



ORIGINAL AWS SCENARIO / COMMAND

```
Review: compute, IP addresses, concurrency and APIs
Request increases early where supported
Load test: throttling and backpressure behavior
DR: target Region needs sufficient headroom too
```

WHAT HAPPENS · Operational capacity includes the ability to create replacement resources under stress.

ORIGINAL INTERVIEWER FOLLOW-UP

Which quota could prevent a failover from launching enough tasks?

Answer: Check Fargate vCPU quotas, ECS or ELB capacity and IP availability for the failover Region and desired scale. Request quota changes before a drill, not during an outage.

Common mistake: Having a quota increase approved does not guarantee instant underlying capacity in every AZ.

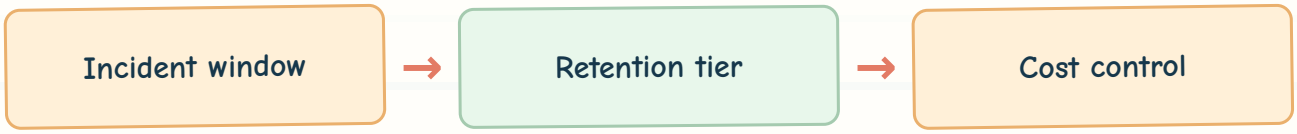
Source guide p. 58 · Further reading: A41, A51, A61

Q089. How do you reduce observability and storage cost safely?

SAY THIS FIRST · SIMPLE ENGLISH

Keep enough logs and metrics to investigate incidents without paying to retain everything forever.

Precise interview detail: Control log volume and retention, sample traces appropriately, avoid high-cardinality metrics and apply storage lifecycle policies consistent with recovery requirements. Keep the audit and diagnostic evidence that the business actually needs.



ORIGINAL AWS SCENARIO / COMMAND

```
Keep: security and critical business audit evidence
Reduce: noisy debug logs and repeated payload dumps
Retain: defined time windows by data purpose
Verify: incident diagnosis and restore still work
```

WHAT HAPPENS · Cost reduction should preserve required visibility and recoverability.

ORIGINAL INTERVIEWER FOLLOW-UP

How would you choose a log-retention period for incident investigation?

Answer: Start from incident detection lag, regulatory needs and forensic lookback. Use different retention for hot logs, archives, traces and sensitive audit events; test that evidence remains searchable.

Common mistake: Deleting all old backups to save money can destroy a recovery objective.

Source guide p. 58 · Further reading: A41, A51, A61

Q090. When is serverless more expensive or less suitable?

SAY THIS FIRST · SIMPLE ENGLISH

Serverless can cost more for high steady traffic, high concurrency, long runtimes, or unsuitable runtime behavior.

Precise interview detail: Serverless value depends on workload shape, latency, duration, concurrency and integrations. Bursty or intermittent workloads can benefit from reduced idle capacity. Sustained compute, expensive invocation patterns or special runtime needs may favor another model.



ORIGINAL AWS SCENARIO / COMMAND

```
Low idle usage + bursts -> evaluate functions
Steady CPU-heavy processing -> compare containers
Heavy DB access -> include connection and proxy cost
Decision -> measured workload and full cost model
```

WHAT HAPPENS · Compare the whole system, including queue, gateway, networking and monitoring.

ORIGINAL INTERVIEWER FOLLOW-UP

What measurements would you gather before migrating a busy Java API to Lambda?

Answer: Measure real request rate, duration, cold/warm latency, memory, concurrency, existing container utilization, API Gateway fees, network costs, and required operations.

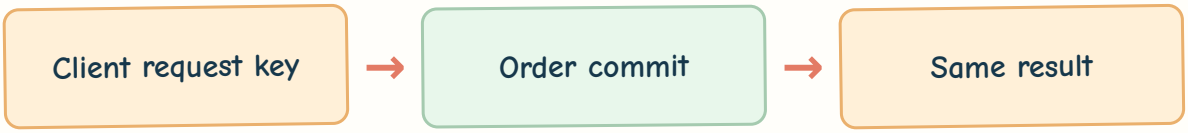
Common mistake: Invocation price alone is not the total architecture cost.

Source guide p. 59 · Further reading: A41, A51, A61

CHAPTER 19 / 20

Senior architecture scenarios

Explain one successful request and one failure.



YOUR FIVE INTERVIEW QUESTIONS

091. Design a resilient Spring Boot order API on AWS.

092. Design a secure file-upload and processing workflow.

093. Design a flash-sale inventory system.

094. Design a multi-tenant reporting service.

095. How would you lead an AWS migration from an on-premises Java monolith?

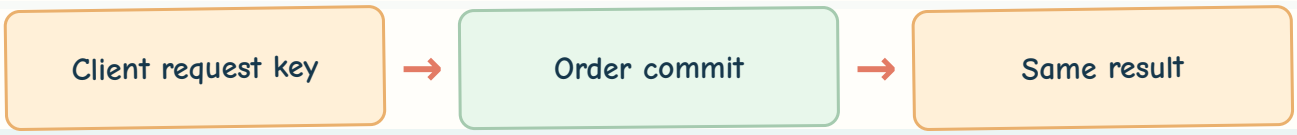
> Say the guarantee and its limit before describing any advanced service.

Q091. Design a resilient Spring Boot order API on AWS.

SAY THIS FIRST · SIMPLE ENGLISH

For an order system, route requests through managed ingress to multi-AZ Java tasks; save orders and outbox events in one transaction; process side effects asynchronously.

Precise interview detail: Use managed ingress, multi-AZ stateless compute, a durable relational store for transactional invariants, and an outbox-driven asynchronous path. Add least-privilege identities, controlled secrets, telemetry and tested recovery. Choose service variants based on explicit traffic and recovery targets.



ORIGINAL AWS SCENARIO / COMMAND

```
Route 53 -> ALB -> ECS Fargate tasks in two AZs
Tasks -> RDS writer -> order + outbox transaction
Publisher -> SQS -> idempotent fulfilment workers
S3 -> private invoices, authorized downloads
```

WHAT HAPPENS · The synchronous response acknowledges the database outcome; asynchronous fulfilment has separate state and latency.

ORIGINAL INTERVIEWER FOLLOW-UP

What do you return if the client times out after the order committed?

Answer: Use a client-supplied request key: when the retry arrives, find the already-committed order and return its stored status or resource. A timeout does not prove rollback.

Common mistake: Do not claim that this topology alone guarantees zero downtime or zero lost orders.

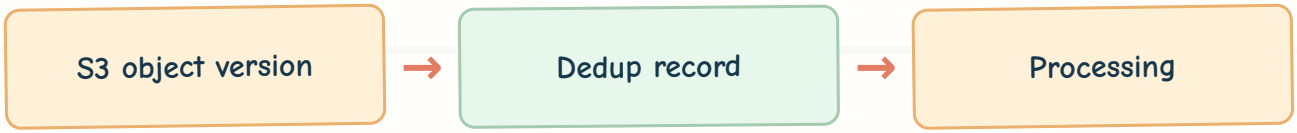
Source guide p. 60 · Further reading: A19, A25, A33, A35, A42

Q092. Design a secure file-upload and processing workflow.

SAY THIS FIRST · SIMPLE ENGLISH

Issue a narrow S3 upload URL, store upload intent, validate the object, then process it asynchronously and publish a safe result.

Precise interview detail: Authorize the uploader, allocate a scoped object key, issue a suitable presigned upload capability and validate the resulting object before use. Trigger bounded processing through a replay-tolerant event path and publish a result only after validation succeeds.



ORIGINAL AWS SCENARIO / COMMAND

```
API -> upload intent and scoped key
Client -> S3 upload
Event -> queue -> scan and transform worker
Status -> pending, accepted, rejected or failed
```

WHAT HAPPENS · The object should remain untrusted until validation, size/type checks and business ownership are confirmed.

ORIGINAL INTERVIEWER FOLLOW-UP

How would you avoid reprocessing the same uploaded object version twice?

Answer: Deduplicate on bucket, object key, and immutable version ID or verified content identity, committing that marker with the processing result. Redeliveries then return the stored outcome.

Common mistake: Treating Content-Type from a client as proof of file content is unsafe.

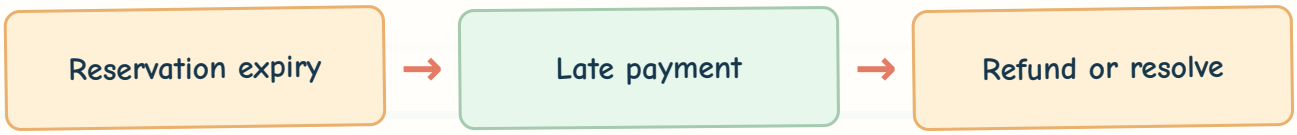
Source guide p. 60 · Further reading: A19, A25, A33, A35, A42

Q093. Design a flash-sale inventory system.

SAY THIS FIRST · SIMPLE ENGLISH

Reserve scarce stock with an atomic database update, put payment in a recoverable state, and use durable request IDs.

Precise interview detail: Make inventory reservation a durable conditional operation or transaction at the chosen authority. Queue bounded downstream work, use stable request IDs and release expired reservations safely. Cache product views while keeping scarce-stock invariants out of ordinary stale caches.



ORIGINAL AWS SCENARIO / COMMAND

```
Attempt reserve -> atomic invariant check
Reservation -> pending with expiry and ownership
Payment outcome -> confirm or release
Duplicate request -> resolve stored outcome
```

WHAT HAPPENS · Admission control keeps backend work within tested capacity even when public demand is much larger.

ORIGINAL INTERVIEWER FOLLOW-UP

How would you reconcile a late successful payment after a reservation expired?

Answer: Check provider outcome and reservation history. If stock was released and cannot safely be reallocated, refund or fulfill through a documented exception workflow; never quietly oversell.

Common mistake: A cache count followed by a database decrement can oversell under concurrent requests.

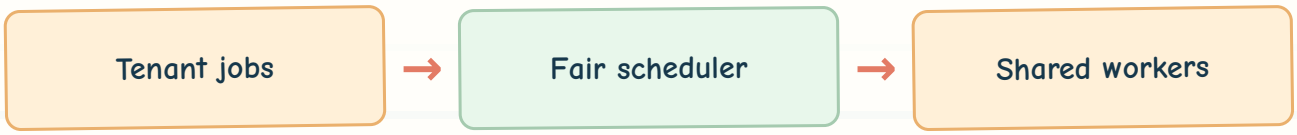
Source guide p. 61 · Further reading: A19, A25, A33, A35, A42

Q094. Design a multi-tenant reporting service.

SAY THIS FIRST · SIMPLE ENGLISH

Queue authorized report jobs, isolate tenant data, and cap workers so a large tenant cannot starve others.

Precise interview detail: Separate authorization from execution scheduling, store input/output under tenant-scoped identities or enforced paths, queue jobs and isolate noisy workloads. Decide fairness, retention, audit and cancellation behavior. Use durable job state so retries do not publish conflicting reports.



ORIGINAL AWS SCENARIO / COMMAND

```
API -> tenant-authorized job record
Queue -> bounded worker concurrency
Worker -> scoped data read -> S3 report object
Download -> verify job ownership -> presigned GET
```

WHAT HAPPENS · Job identity and tenant identity must travel through the asynchronous boundary.

ORIGINAL INTERVIEWER FOLLOW-UP

How would you enforce per-tenant fairness without a queue for every customer?

Answer: Use tenant-aware scheduling with per-tenant active-job limits or weighted fair queues over a shared worker pool. Persist tenant identity with each job and apply global safety limits.

Common mistake: One giant tenant should not consume all workers and starve every other tenant.

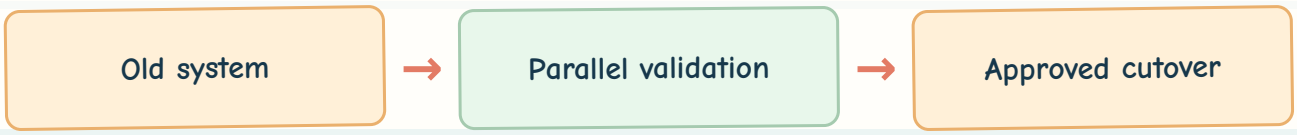
Source guide p. 61 · Further reading: A19, A25, A33, A35, A42

Q095. How would you lead an AWS migration from an on-premises Java monolith?

SAY THIS FIRST · SIMPLE ENGLISH

Migrate a monolith in measured steps: inventory dependencies, establish networking and identity, pilot a small workload, rehearse data cutover, then stabilize.

Precise interview detail: Inventory dependencies, data and operational requirements, establish a secure network and identity foundation, then migrate a bounded workload with measurable parity. Choose rehosting, replatforming or redesign per component. Plan data cutover, rollback and operational ownership before execution.



ORIGINAL AWS SCENARIO / COMMAND

```
Assess -> baseline performance and dependency map
Pilot -> bounded environment and rehearsed data move
Cutover -> controlled writes, verification, rollback rule
Optimize -> measured improvements after stabilization
```

WHAT HAPPENS · A migration is complete only when the team can operate, recover and control cost in the new environment.

ORIGINAL INTERVIEWER FOLLOW-UP

What evidence would allow the business to approve the data cutover?

Answer: Require tested reconciliation totals, business transaction parity, performance metrics, cutover rehearsal, rollback checkpoints, security sign-off, and a clear owner for go/no-go.

Common mistake: Rewriting every component during the migration can combine too many unproven changes.

Source guide p. 62 · Further reading: A19, A25, A33, A35, A42

CHAPTER 20 / 20

Troubleshooting and interview traps

Diagnose the faulty layer before changing things.



YOUR FIVE INTERVIEW QUESTIONS

096. An ECS task cannot pull its image. What do you check?

097. SQS queue age rises while CPU stays low. What might be wrong?

098. An S3 request returns AccessDenied. What is your diagnosis?

099. A deployment is healthy but customers get errors. Why?

100. Which AWS interview claims should you avoid?

> Say the guarantee and its limit before describing any advanced service.

Q096. An ECS task cannot pull its image. What do you check?

SAY THIS FIRST · SIMPLE ENGLISH

For failed ECS image pulls, check task stop reasons, execution role, image digest, registry path, CPU architecture, and network route.

Precise interview detail: Inspect the task stopped reason, image reference, execution-role permissions, registry availability and network paths to required endpoints. Check platform architecture and credentials where applicable. Application task-role permissions are a different path.



ORIGINAL AWS SCENARIO / COMMAND

```
Evidence: stopped reason and container reason
Identity: execution role for supported ECR pull
Network: routes, DNS, security and endpoints/NAT
Artifact: repository, digest, architecture
```

WHAT HAPPENS · The failure occurs before your application starts, so application logs may be absent.

ORIGINAL INTERVIEWER FOLLOW-UP

Which endpoints might a private-subnet ECR pull depend on?

Answer: A private-subnet pull from ECR commonly needs connectivity to ECR API and Docker registry endpoints and the S3 layer-storage path, through endpoints or NAT as designed. Verify DNS and security groups.

Common mistake: Debugging JDBC settings will not repair a missing image-pull route.

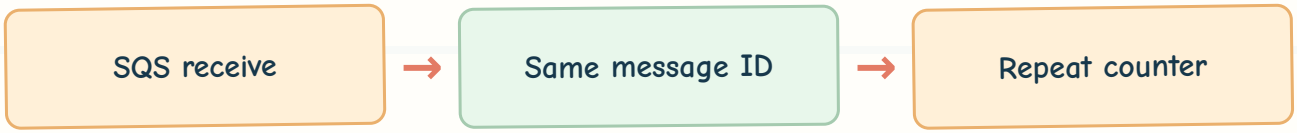
Source guide p. 63 · Further reading: A03, A17, A20, A33, A43

Q097. SQS queue age rises while CPU stays low. What might be wrong?

SAY THIS FIRST · SIMPLE ENGLISH

Growing SQS message age with low CPU often means workers are waiting on downstream resources, not doing useful work.

Precise interview detail: Inspect consumer concurrency, blocked dependency calls, visibility churn, poison messages and polling configuration. Low CPU can indicate waiting rather than spare useful capacity. Compare processing rate with incoming rate and examine downstream saturation.



ORIGINAL AWS SCENARIO / COMMAND

```
Incoming: 200 messages/s
Processed: 100 messages/s
CPU: 20%, DB pool wait: high
Action: fix waiting bottleneck before more workers
```

WHAT HAPPENS · Scaling consumers can worsen the database queue if every consumer waits on the same constrained resource.

ORIGINAL INTERVIEWER FOLLOW-UP

How would you detect repeated processing of the same message?

Answer: Track message ID, receive count, processing attempts, idempotency-store hits and DLQ events. Compare these with age and throughput to spot visibility churn or repeats.

Common mistake: Backlog size alone cannot explain processing progress without message age and throughput.

Source guide p. 63 · Further reading: A03, A17, A20, A33, A43

Q098. An S3 request returns AccessDenied. What is your diagnosis?

SAY THIS FIRST · SIMPLE ENGLISH

For S3 AccessDenied, check exactly which caller, operation, key, bucket, endpoint policy, SCP and KMS key are involved.

Precise interview detail: Capture the actual caller, operation, bucket/key and encryption context without exposing credentials. Check applicable IAM, bucket, endpoint and organization policies, relevant ownership settings and KMS permissions. Verify the ARN type and request conditions.



ORIGINAL AWS SCENARIO / COMMAND

```
Who: workload role session
What: GetObject, not ListBucket
Where: object ARN and intended prefix
Extra: KMS decrypt and endpoint policy if applicable
```

WHAT HAPPENS · An error can come from several authorization layers; repair the specific missing or denied permission.

ORIGINAL INTERVIEWER FOLLOW-UP

How would you prove the repair still denies another tenant's prefix?

Answer: Test the intended prefix with a least-privileged role and a neighboring tenant's prefix using the same operation. The first should work; the cross-tenant test must remain denied.

Common mistake: Replacing policies with Action:* and Resource:* masks the cause and grants unnecessary authority.

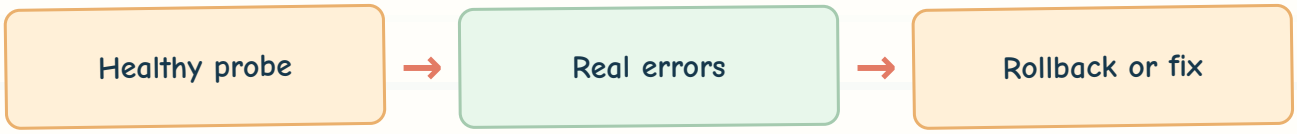
Source guide p. 64 · Further reading: A03, A17, A20, A33, A43

Q099. A deployment is healthy but customers get errors. Why?

SAY THIS FIRST · SIMPLE ENGLISH

A healthy load balancer target may still fail real orders due to auth, database, routing, or schema problems.

Precise interview detail: Health probes may cover only a narrow endpoint or traffic path. Check real business requests, authentication, schema compatibility, dependency access and routing. Compare new and old versions using customer-facing metrics and representative smoke tests.



ORIGINAL AWS SCENARIO / COMMAND

```
Probe: HTTP readiness passes
Business request: order insert fails schema check
Routing: only one path points to the wrong target
Evidence: per-route errors and deployed revision
```

WHAT HAPPENS · Infrastructure health and application correctness are separate observations.

ORIGINAL INTERVIEWER FOLLOW-UP

What should trigger rollback versus a forward fix?

Answer: Rollback when a new version causes widespread user-visible failures and old code remains data-compatible. Forward-fix a small isolated defect only if risk and time are lower and rollback cannot solve it.

Common mistake: A green deployment dashboard is not sufficient release evidence.

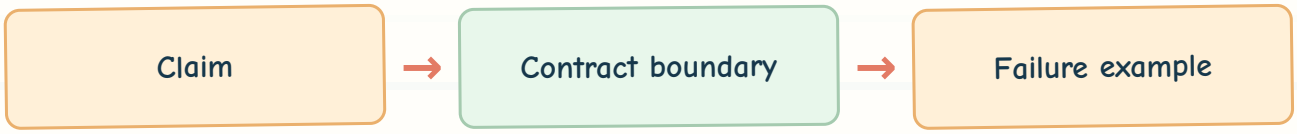
Source guide p. 64 · Further reading: A03, A17, A20, A33, A43

Q100. Which AWS interview claims should you avoid?

SAY THIS FIRST · SIMPLE ENGLISH

Avoid oversimplified statements such as 'managed means secure' or 'FIFO means exactly once.' Always state the actual scope and failure case.

Precise interview detail: Avoid guarantees broader than the service contract or your validation. State scope, topology and failure assumptions. Distinguish deployment configuration from observed production behavior and measured recovery.



ORIGINAL AWS SCENARIO / COMMAND

```
Avoid: 'FIFO makes payment exactly once'
Avoid: 'all Multi-AZ standbys are identical'
Avoid: 'a private subnet has no outbound access'
Avoid: 'managed means no customer responsibility'
```

WHAT HAPPENS · Replace slogans with one precise statement and a failure example.

ORIGINAL INTERVIEWER FOLLOW-UP

Pick one claim and defend a correct replacement in 60 seconds.

Answer: For example: SQS FIFO can constrain message order and deduplication within its contract, but a payment consumer still needs an idempotency key because a message or external effect may be retried.

Common mistake: Memorizing service names without explaining their limits weakens senior-level answers.

Source guide p. 65 · Further reading: A03, A17, A20, A33, A43

WORKED ARCHITECTURE

Spring Boot Orders on AWS

A durable order, a recoverable event, and an idempotent worker.



Normal path: A client submits a stable request ID. The Java service checks stock and writes the order plus outbox event in one database transaction. A publisher sends the event to SQS. A worker applies an idempotent fulfilment effect.

Failure path: If the client loses a successful response, the retry returns the stored order instead of creating another. If the publisher resends, the consumer detects the duplicate.

Example only. Replace accounts, Regions, IAM roles, image digests, secrets, networks and database settings. The example has not been deployed to AWS here.

ARCHITECTURE DECISIONS

Separate the boundaries

Ingress: ALB terminates HTTPS and forwards to healthy targets; WAF only where needed.

Network: ALB uses intended public placement; API tasks and DB use approved private placements and scoped security groups.

Identity: Execution role handles image pull/logs/injected secrets; task role handles app SDK calls; worker has separate rights.

Durable order: One RDS transaction commits order and outbox so event intent cannot disappear after an acknowledged order.

Messages: SQS can redeliver. Worker saves a durable event key before committing local effect.

Observability: Monitor p99, target health, pool waits, outbox age, queue age, rejected operations and restore tests.

WORKED EXAMPLE · SOURCE MATERIAL

ECS role trust policy

Controls who may assume the task role. AWS account and Region are teaching placeholders.

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Principal": {
        "Service": "ecs-tasks.amazonaws.com"
      },
      "Action": "sts:AssumeRole",
      "Condition": {
        "StringEquals": {
          "aws:SourceAccount": "123456789012"
        },
        "ArnLike": {
          "aws:SourceArn": "arn:aws:ecs:us-east-1:123456789012:*"
        }
      }
    }
  ]
}
```

> This is an illustration, not evidence of successful deployment or service authorization.

WORKED EXAMPLE · SOURCE MATERIAL

Application task permissions

One S3 prefix plus one SQS queue. It does not include unrelated actions.

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": [
        "s3:GetObject"
      ],
      "Resource": [
        "arn:aws:s3:::EXAMPLE_BUCKET/reports/*"
      ]
    },
    {
      "Effect": "Allow",
      "Action": [
        "sqs:SendMessage"
      ],
      "Resource": [
        "arn:aws:sqs:us-east-1:123456789012:EXAMPLE_QUEUE"
      ]
    }
  ]
}
```

> This is an illustration, not evidence of successful deployment or service authorization.

WORKED EXAMPLE · SOURCE MATERIAL

Fargate task definition

Reviewed image, roles, actual networking and safe secrets are required before deployment.

```
{
  "family": "interview-orders",
  "networkMode": "awsvpc",
  "requiresCompatibilities": [
    "FARGATE"
  ],
  "cpu": "512",
  "memory": "1024",
  "runtimePlatform": {
    "cpuArchitecture": "X86_64",
    "operatingSystemFamily": "LINUX"
  },
  "taskRoleArn": "arn:aws:iam::123456789012:role/EXAMPLE_TASK_ROLE",
  "executionRoleArn": "arn:aws:iam::123456789012:role/EXAMPLE_EXECUTION_ROLE",
  "containerDefinitions": [
    {
      "name": "orders",
      "image": "REPLACE_WITH_REVIEWED_IMAGE_AT_DIGEST",
      "essential": true,
      "stopTimeout": 30,
      "portMappings": [
        {
          "containerPort": 8080,
          "protocol": "tcp"
        }
      ],
      "environment": [
        {
          "name": "SPRING_PROFILES_ACTIVE",
          "value": "prod"
        },
        {
          "name": "SPRING_DATASOURCE_URL",
          "value": "REPLACE_WITH_JDBC_URL"
        }
      ],
      "secrets": [
        {
          "name": "SPRING_DATASOURCE_PASSWORD",
          "valueFrom": "arn:aws:secretsmanager:us-east-1:123456789012:secret:EXAMPLE_SECRET"
        }
      ],
      "logConfiguration": {
        "logDriver": "awslogs",
        "options": {
          "awslogs-group": "/interview/orders",
          "awslogs-region": "us-east-1",
          "awslogs-stream-prefix": "ecs"
        }
      }
    }
  ]
}
```

> This is an illustration, not evidence of successful deployment or service authorization.

WORKED EXAMPLE · SOURCE MATERIAL

Spring Boot properties

Health endpoints need correct Actuator and authentication settings.

```
server.shutdown=graceful
spring.lifecycle.timeout-per-shutdown-phase=20s
management.endpoint.health.probes.enabled=true
management.endpoints.web.exposure.include=health,info
# Configure datasource and secrets externally.
```

> This is an illustration, not evidence of successful deployment or service authorization.

WORKED EXAMPLE · SOURCE MATERIAL

AWS SDK Java 17 source

Uses SDK v2 default credential chain and headObject to read object metadata.

```
package interview;
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.s3.S3Client;
import software.amazon.awssdk.services.s3.model.HeadObjectRequest;
import software.amazon.awssdk.services.s3.model.HeadObjectResponse;
public final class ObjectMetadata {
    public static void main(String[] args) {
        if (args.length != 3) {
            throw new IllegalArgumentException(
                "Usage: ObjectMetadata REGION BUCKET KEY");
        }
        try (S3Client s3 = S3Client.builder()
                .region(Region.of(args[0])).build()) {
            HeadObjectResponse result = s3.headObject(
                HeadObjectRequest.builder()
                    .bucket(args[1]).key(args[2]).build());
            System.out.println("bytes=" + result.contentLength());
            System.out.println("version=" + result.versionId());
        }
    }
}
```

> This is an illustration, not evidence of successful deployment or service authorization.

WORKED EXAMPLE · SOURCE MATERIAL

Maven project definition

Replace the SDK BOM placeholder with a reviewed available 2.x release before building.

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
         https://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>
  <groupId>interview</groupId>
  <artifactId>aws-metadata</artifactId>
  <version>1.0.0</version>
  <properties>
    <maven.compiler.release>17</maven.compiler.release>
    <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
    <aws.sdk.version>REPLACE_WITH_REVIEWED_2_X_VERSION</aws.sdk.version>
  </properties>
  <dependencyManagement>
    <dependencies>
      <dependency>
        <groupId>software.amazon.awssdk</groupId>
        <artifactId>bom</artifactId>
        <version>${aws.sdk.version}</version>
        <type>pom</type><scope>import</scope>
      </dependency>
    </dependencies>
  </dependencyManagement>
  <dependencies>
    <dependency>
      <groupId>software.amazon.awssdk</groupId>
      <artifactId>s3</artifactId>
    </dependency>
  </dependencies>
  <build><plugins><plugin>
    <groupId>org.apache.maven.plugins</groupId>
    <artifactId>maven-compiler-plugin</artifactId>
    <version>3.14.0</version>
  </plugin></plugins></build>
</project>
```

> This is an illustration, not evidence of successful deployment or service authorization.

HANDS-ON PRACTICE

8 AWS exercises

Four safe read-only cloud inspections. Four tested local simulations.

Verify AWS identity → Inspect actual contract → Explain result

L01–L04: AWS CLI v2, authorized profile and existing resources. No AWS API calls were made while producing this guide.

L05–L08: Python 3 offline exercises for IAM structure, connection budgeting, duplicate delivery and task JSON.

Run cloud commands only in an account you own or are authorized to inspect. Check identity and Region first; use read-only permissions.

HANDS-ON EXERCISE 01 / 08

L01. Confirm the identity and Region before diagnosis

This identifies a caller; it does not prove the caller may read S3 or deploy ECS. No resources are created or deleted.

BASH SCRIPT FROM THE ATTACHED GUIDE

```
set -euo pipefail
export AWS_PAGER=""
PROFILE=YOUR_PROFILE
REGION=YOUR_REGION
aws --profile "$PROFILE" --region "$REGION" \
  sts get-caller-identity --output json
aws --profile "$PROFILE" configure get region
```

Expected: The identity response contains Account, Arn and UserId. Confirm they match the intended environment. The configured Region can be empty or differ from the explicit REGION, which overrides it for the first command.

Verification: Syntax verified; requires AWS account. AWS account permissions, API behavior and production readiness were not checked.

HANDS-ON EXERCISE 02 / 08

L02. Inspect an existing S3 bucket without reading object contents

ETag is not a universal object-content MD5. Account-level public-access controls and bucket policies also matter. The script stops on the first API failure so inspect that failure before continuing.

BASH SCRIPT FROM THE ATTACHED GUIDE

```
set -euo pipefail
export AWS_PAGER=""
PROFILE=YOUR_PROFILE
REGION=YOUR_REGION
BUCKET=YOUR_BUCKET
KEY=YOUR_EXISTING_OBJECT_KEY
aws --profile "$PROFILE" --region "$REGION" \
  s3api get-bucket-versioning --bucket "$BUCKET"
aws --profile "$PROFILE" --region "$REGION" \
  s3api get-public-access-block --bucket "$BUCKET"
aws --profile "$PROFILE" --region "$REGION" \
  s3api head-object --bucket "$BUCKET" --key "$KEY" \
  --query '{Size:ContentLength,ETag:ETag,Version:VersionId}'
```

Expected: Record the versioning state, public-access settings and object metadata. An absent versioning Status differs from Enabled or Suspended. A missing public-access-block configuration can produce an API error; do not infer public exposure from that error alone.

Verification: Syntax verified; requires AWS account. AWS account permissions, API behavior and production readiness were not checked.

HANDS-ON EXERCISE 03 / 08

L03. Investigate a known ECS task and service

A describe response is a snapshot rather than proof of ongoing health. Correlate timestamps with target health, deployment identity and application errors.

BASH SCRIPT FROM THE ATTACHED GUIDE

```
set -euo pipefail
export AWS_PAGER=""
PROFILE=YOUR_PROFILE
REGION=YOUR_REGION
CLUSTER=YOUR_CLUSTER
SERVICE=YOUR_SERVICE
TASK_ARN=YOUR_TASK_ARN
aws --profile "$PROFILE" --region "$REGION" \
  ecs describe-tasks --cluster "$CLUSTER" \
  --tasks "$TASK_ARN" \
  --query '{Tasks:tasks,Failures:failures}'
aws --profile "$PROFILE" --region "$REGION" \
  ecs describe-services --cluster "$CLUSTER" \
  --services "$SERVICE" \
  --query '{Services:services,Failures:failures}'
```

Expected: Locate taskDefinitionArn, lastStatus, stopCode/stoppedReason when available, container reasons, network attachments and service deployment events. Check failures arrays even if the command exits successfully.

Verification: Syntax verified; requires AWS account. AWS account permissions, API behavior and production readiness were not checked.

HANDS-ON EXERCISE 04 / 08

L04. Read an SQS queue processing contract

Approximate counts are not exact accounting. No live consumer or retry behavior is tested by this metadata request.

BASH SCRIPT FROM THE ATTACHED GUIDE

```
set -euo pipefail
export AWS_PAGER=""
PROFILE=YOUR_PROFILE
REGION=YOUR_REGION
QUEUE_URL=YOUR_QUEUE_URL
aws --profile "$PROFILE" --region "$REGION" \
  sqs get-queue-attributes --queue-url "$QUEUE_URL" \
  --attribute-names VisibilityTimeout \
  MessageRetentionPeriod RedrivePolicy \
  ApproximateNumberOfMessages \
  ApproximateNumberOfMessagesNotVisible
```

Expected: Explain visibility duration, retention and any redrive policy. Compare approximate visible and in-flight counts with processing rate and oldest-message age from operational metrics. An unset redrive policy is not a DLQ configuration.

Verification: Syntax verified; requires AWS account. AWS account permissions, API behavior and production readiness were not checked.

HANDS-ON EXERCISE 05 / 08

L05. Check the structure of a prefix-scoped IAM policy

No AWS policy simulator or account authorization check is performed. A structurally valid allow can still be denied by other applicable policies.

BASH SCRIPT FROM THE ATTACHED GUIDE

```
python3 - <<'PY'
import json
policy = {
 "Version": "2012-10-17",
 "Statement": [{
  "Effect": "Allow",
  "Action": ["s3:GetObject"],
  "Resource": ["arn:aws:s3:::example-interview/reports/*"]
 }]
}
encoded = json.dumps(policy, indent=2)
decoded = json.loads(encoded)
s = decoded["Statement"][0]
assert s["Action"] == ["s3:GetObject"]
assert s["Resource"] == [
 "arn:aws:s3:::example-interview/reports/*"
]
assert "Principal" not in s
print(encoded)
print("PASS: intended identity-policy statement")
PY
```

Expected: The script prints valid JSON and a PASS line. Explain why ListBucket and KMS Decrypt are absent and would require separate applicable permissions if the operation needed them.

Verification: Syntax verified; offline executed. AWS account permissions, API behavior and production readiness were not checked.

HANDS-ON EXERCISE 06 / 08

L06. Calculate a deployment-wide JDBC connection budget

A connection budget is not a throughput estimate. Query time, locks, proxy pinning and other sessions can reduce safe concurrency.

BASH SCRIPT FROM THE ATTACHED GUIDE

```
python3 - <<'PY'
db_limit = 300
reserved_for_ops = 30
other_apps = 50
max_tasks = 20
pool_per_task = 15
budget = db_limit - reserved_for_ops - other_apps
worst_case = max_tasks * pool_per_task
safe_pool = budget // max_tasks
assert budget == 220
assert worst_case == 300
assert safe_pool == 11
print("Application budget:", budget)
print("Requested worst case:", worst_case)
print("Upper bound per task:", safe_pool)
assert worst_case > budget
print("PASS: proposed pool exceeds budget")
PY
```

Expected: The application budget is 220, requested worst-case connections are 300 and the arithmetic upper bound is 11 per task. This bound still requires workload and failover headroom validation.

Verification: Syntax verified; offline executed. AWS account permissions, API behavior and production readiness were not checked.

HANDS-ON EXERCISE 07 / 08

L07. Simulate duplicate delivery with a durable invariant

This models a local database effect only. It does not prove exactly-once payment, queue delivery or cross-service atomicity. An external effect needs its own idempotency/ reconciliation contract.

BASH SCRIPT FROM THE ATTACHED GUIDE

```
python3 - <<'PY'
import sqlite3
c = sqlite3.connect(":memory:")
c.executescript("""
CREATE TABLE orders(id TEXT PRIMARY KEY);
CREATE TABLE outbox(event_id TEXT PRIMARY KEY,
                    order_id TEXT NOT NULL);
CREATE TABLE effects(event_id TEXT PRIMARY KEY,
                     order_id TEXT NOT NULL);
""")
with c:
 c.execute("INSERT INTO orders VALUES (?)", ("O42",))
 c.execute("INSERT INTO outbox VALUES (?, ?)",
           ("E42", "O42"))
event = c.execute("SELECT * FROM outbox").fetchone()
for delivery in range(2):
 with c:
  c.execute("INSERT OR IGNORE INTO effects VALUES (?, ?)",
            event)
count = c.execute("SELECT count(*) FROM effects").fetchone()[0]
assert count == 1
assert c.execute("SELECT count(*) FROM orders").fetchone()[0] == 1
print("PASS: two deliveries, one durable local effect")
PY
```

Expected: Two deliveries yield one effects row because event_id is unique. The initial order and outbox records are committed together. Explain how a real publisher can resend after a send-success/mark-failure window.

Verification: Syntax verified; offline executed. AWS account permissions, API behavior and production readiness were not checked.

HANDS-ON EXERCISE 08 / 08

L08. Validate a minimal Fargate task-definition example

This is not an AWS API validation, complete security review or resource provisioning test. Fargate platform and task parameters must match the chosen deployment.

BASH SCRIPT FROM THE ATTACHED GUIDE

```
python3 - <<'PY'
import json
spec = {
 "family":"interview-orders",
 "networkMode":"awsvpc",
 "requiresCompatibilities":["FARGATE"],
 "cpu":"512", "memory":"1024",
 "containerDefinitions":[{
  "name":"orders", "image":"REPLACE_WITH_REVIEWED_IMAGE",
  "essential":True,
  "portMappings":[{"containerPort":8080,"protocol":"tcp"}]
 }]
}
x = json.loads(json.dumps(spec))
assert x["networkMode"] == "awsvpc"
assert x["cpu"] == "512" and x["memory"] == "1024"
assert x["containerDefinitions"][0]["essential"] is True
assert x["containerDefinitions"][0]["portMappings"][0]["containerPort"] == 8080
print(json.dumps(x, indent=2))
print("PASS: intended minimal example fields")
PY
```

Expected: The script prints the task-definition JSON and a PASS line. Identify what remains before deployment: real image, roles, network placement, ingress, logs, secrets and application readiness.

Verification: Syntax verified; offline executed. AWS account permissions, API behavior and production readiness were not checked.

REVISION CARD

Which AWS service?

Public HTTP routing	**ALB or API Gateway**
Containers with AWS-native workflow	**ECS; Fargate where appropriate**
Kubernetes ecosystem	**EKS**
Short event computation	**Lambda**
Relational consistency	**RDS / Aurora**
Object uploads	**S3 + metadata DB**
Key-value ordered access	**DynamoDB**
Cache read-heavy data	**ElastiCache**

REVISION CARD · PART 2

Data, events & operations

Distribute jobs	**SQS**
Publish to subscribers	**SNS + subscriber queues**
Route business events	**EventBridge**
Retained ordered streams	**Kinesis or MSK**
Workflow coordination	**Step Functions**
Manage secrets	**Secrets Manager / SSM**
Metrics and alarms	**CloudWatch**
AWS API audit	**CloudTrail**

Technology names do not prove availability, correct permissions or durability. Define what the operation must guarantee.

INVESTIGATE SYMPTOMS

Signal → first evidence

ECS image cannot pull
Stopped reason, execution role, ECR/S3 endpoint path, image platform.

ECS task starts but API fails
Task logs, ALB target health, Spring listener, secrets and JDBC.

SQS backlog age grows
Arrival/processing rate, DB pool waits, redeliveries and oldest age.

Sudden p99 latency
Distributed trace, pool wait, query duration, throttling and deploy history.

S3 AccessDenied
Actual caller, GetObject versus ListBucket, bucket and KMS policy.

RDS writer fails over
Connection refresh, commit uncertainty, request idempotency.

Cache outage causes DB spike
Cache miss rate, fallback concurrency caps and DB load.

Rollout healthy but orders fail
Smoke tests, schema compatibility, business outcome metrics.

High AWS bill
Cost Explorer dimensions, transfer, NAT, logs, storage and usage spikes.

14-DAY PREPARATION PLAN

Practice with purpose

1–2 Cloud responsibility, Regions, IAM and VPC; draw a network and solve L01.

3–4 EC2, S3 and edge; test S3 read permissions using L02 if authorized.

5–6 ECS/EKS, Java startup and Lambda; discuss L03.

7–8 RDS, DynamoDB and cache; complete JDBC budget L06.

9–10 SQS, streaming and workflows; perform L04/L07.

11 Observability, Secrets Manager, KMS and tenant isolation; L05.

12 IaC, CI/CD, resilience and quotas; L08.

13 Sketch the complete Spring Boot orders architecture.

14 Mock 45-minute system design, 10 random follow-ups and one failure drill.

Senior interview method: state the invariant → name the owning service → trace success → inject failure → explain validation and cost.

REFERENCES

Official AWS documentation

Source-guide links are retained for checking service details. No new online verification was performed for this edition; check the live AWS docs before production use.

https://aws.amazon.com/compliance/shared-responsibility-model/

https://aws.amazon.com/about-aws/global-infrastructure/regions_az/

https://docs.aws.amazon.com/IAM/latest/UserGuide/reference_policies_evaluation-logic.html

https://docs.aws.amazon.com/IAM/latest/UserGuide/reference_policies_evaluation-logic_policy-eval-denyallow.html

https://docs.aws.amazon.com/IAM/latest/UserGuide/id_roles.html

https://docs.aws.amazon.com/organizations/latest/userguide/orgs_manage_policies_scps.html

https://docs.aws.amazon.com/vpc/latest/userguide/vpc-example-private-subnets-nat.html

https://docs.aws.amazon.com/vpc/latest/userguide/vpc-security-groups.html

https://docs.aws.amazon.com/vpc/latest/userguide/vpc-network-acls.html

https://docs.aws.amazon.com/vpc/latest/privatelink/concepts.html

https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/ec2-instance-lifecycle.html

https://docs.aws.amazon.com/autoscaling/ec2/userguide/what-is-amazon-ec2-auto-scaling.html

https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/instance-purchasing-options.html

https://docs.aws.amazon.com/AmazonS3/latest/userguide/Welcome.html

https://docs.aws.amazon.com/AmazonS3/latest/userguide/using-presigned-url.html

https://docs.aws.amazon.com/AmazonS3/latest/userguide/Versioning.html

https://docs.aws.amazon.com/elasticloadbalancing/latest/application/target-group-health-checks.html

https://docs.aws.amazon.com/AmazonCloudFront/latest/DeveloperGuide/controlling-the-cache-key.html

https://docs.aws.amazon.com/AmazonECS/latest/developerguide/task-iam-roles.html

https://docs.aws.amazon.com/AmazonECS/latest/developerguide/task_execution_IAM_role.html

https://docs.aws.amazon.com/eks/latest/userguide/security.html

https://docs.aws.amazon.com/lambda/latest/dg/invocation-retries.html

https://docs.aws.amazon.com/lambda/latest/dg/lambda-concurrency.html

https://docs.aws.amazon.com/lambda/latest/dg/snapstart.html

https://docs.aws.amazon.com/AmazonRDS/latest/UserGuide/Concepts.MultiAZSingleStandby.html

https://docs.aws.amazon.com/AmazonRDS/latest/UserGuide/multi-az-db-clusters-concepts.html

https://docs.aws.amazon.com/AmazonRDS/latest/UserGuide/rds-proxy.html

https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/HowItWorks.ReadConsistency.html

https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/V2globaltables_HowItWorks.html

https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/TTL.html

https://docs.aws.amazon.com/AWSSimpleQueueService/latest/SQSDeveloperGuide/standard-queues.html

https://docs.aws.amazon.com/AWSSimpleQueueService/latest/SQSDeveloperGuide/FIFO-queues-exactly-once-processing.html

https://docs.aws.amazon.com/AWSSimpleQueueService/latest/SQSDeveloperGuide/sqs-visibility-timeout.html

https://docs.aws.amazon.com/AWSSimpleQueueService/latest/SQSDeveloperGuide/sqs-dead-letter-queues.html

https://docs.aws.amazon.com/prescriptive-guidance/latest/cloud-design-patterns/transactional-outbox.html

https://docs.aws.amazon.com/step-functions/latest/dg/choosing-workflow-type.html

https://docs.aws.amazon.com/kms/latest/developerguide/concepts.html

https://docs.aws.amazon.com/secretsmanager/latest/userguide/rotating-secrets.html

https://docs.aws.amazon.com/awscloudtrail/latest/userguide/logging-data-events-with-cloudtrail.html

https://docs.aws.amazon.com/config/latest/developerguide/WhatIsConfig.html

https://docs.aws.amazon.com/wellarchitected/latest/framework/welcome.html

https://docs.aws.amazon.com/whitepapers/latest/disaster-recovery-workloads-on-aws/disaster-recovery-options-in-the-cloud.ht

https://docs.aws.amazon.com/AmazonECS/latest/developerguide/task_definition_parameters.html

https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/credentials-chain.html

https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/java_s3_code_examples.html

https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html

https://docs.aws.amazon.com/AWSCloudFormation/latest/UserGuide/using-cfn-updating-stacks-changesets.html

https://docs.aws.amazon.com/IAM/latest/UserGuide/id_roles_providers_create_oidc.html

https://docs.aws.amazon.com/AmazonECS/latest/developerguide/secrets-envvar-secrets-manager.html

https://docs.spring.io/spring-boot/reference/web/graceful-shutdown.html

https://docs.aws.amazon.com/AmazonElastiCache/latest/dg/WhatIs.deployment.html

https://docs.aws.amazon.com/decision-guides/latest/decision-guides/sns-or-sqs-or-eventbridge.html

https://docs.aws.amazon.com/directconnect/latest/UserGuide/encryption-in-transit.html

https://docs.aws.amazon.com/vpc/latest/peering/invalid-peering-configurations.html

https://docs.aws.amazon.com/Route53/latest/DeveloperGuide/routing-policy.html

https://docs.aws.amazon.com/apigateway/latest/developerguide/http-api-vs-rest.html

https://docs.aws.amazon.com/streams/latest/dev/key-concepts.html

https://docs.aws.amazon.com/msk/latest/developerguide/what-is-msk.html

https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/Expressions.ConditionExpressions.html

https://docs.aws.amazon.com/systems-manager/latest/userguide/systems-manager-parameter-store.html

https://docs.aws.amazon.com/servicequotas/latest/userguide/intro.html

https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/configuring-instance-metadata-options.html

https://maven.apache.org/plugins-archives/maven-compiler-plugin-3.14.0/

https://docs.aws.amazon.com/waf/latest/developerguide/what-is-aws-waf.html

https://docs.aws.amazon.com/waf/latest/developerguide/shield-chapter.html

https://docs.aws.amazon.com/cognito/latest/developerguide/amazon-cognito-user-pools-using-tokens-verifying-a-jwt.html

https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/Streams.html